TECHNICAL REFERENCE

AgenticMeetingsManager

Backend Architecture Documentation

Configuration, modules, request flows, data handling, security and operations, written from the code as it exists today.

Field	Details
Repository	Agentic-Meetings-Manager (npm package agentic-meetings-manager v1.0.0)
Code snapshot	Branch foundation @ 306d9c0 (2026-08-25) plus uncommitted working-tree changes in src/index.ts, src/config/redis.ts and the new src/middlewares/cors.middleware.ts
Generated	26 October 2026
Verification runtime	Node.js v24.15.0 · TypeScript 7.0.2 (tsc --noEmit passes) · local smoke tests with throwaway configuration
Audience	Backend developers who are new to this repository
Scope	Documentation only. No application source file was modified.
Independence	Standalone project. Nothing in this document is drawn from any other codebase.

READ THIS FIRST

Project stage: foundation. The backend implements user registration and login against an in-memory user store. No meeting-management or agent (LLM) functionality exists in the code yet. The name describes the intended product, not current behaviour.

Table of Contents

1. Cover Page 1

2. Executive Summary 4

2.1 What the backend does today 4
2.2 Most important findings 4
2.3 How to read this document 5

3. Backend Project Overview 6

3.1 Project identity 6
3.2 Version-control history 6
3.3 Branch state: the merged feature is missing from master 6
3.4 Working tree versus the last commit 6
3.5 Capability matrix 7

4. Repository Structure 8

4.1 Directory responsibilities 8
4.2 What is tracked by git 9

5. Complete File-by-File Reference 10

6. Technology Stack and Dependencies 14

6.1 Role of each active technology 14
6.2 Not present 15

7. Backend Setup and Configuration 16

7.1 Prerequisites 16
7.2 Install and run 16
7.3 TypeScript compiler options 16
7.4 How configuration is loaded 17
7.5 Middleware registration and execution order 17
7.6 Dependency wiring 17
7.7 Deployment and containerisation 17
7.8 Configuration reference 17

8. Environment Variables 19

8.1 Environment files 19

9. High-Level Architecture 20

9.1 Layers 20
9.2 Module dependency graph 21

10. Application Startup and Lifecycle 22

10.1 Startup step by step 22
10.2 Ordering subtleties 23
10.3 Graceful shutdown 23

11. API Endpoint Reference 24

11.1 POST /auth/register 24
11.2 POST /auth/login 24
11.3 Cross-cutting API behaviour 25

12. Controller, Service, and Repository Execution Flows 27

12.1 Following a request through the code 27
12.2 Registration flow 28
12.3 Login flow 29
12.4 Error propagation 29

13. Database Architecture and Data Flow 30

13.1 The actual data store 30

13.2 Operations 30
13.3 Consistency, transactions and concurrency 30
13.4 Lifecycle and scope of the data 31
13.5 Dormant persistence artefacts 31
14. Core Business Logic 32
F1 · User registration 32
F2 · Login and access-token issuance 32
F3 · Request authentication middleware (implemented, unused) 32
F4 · Request correlation and access logging 33
F5 · Cross-origin policy 33
F6 · Graceful shutdown 33
15. External Integrations 34
16. Queues, Events, and Background Jobs 35
17. Authentication, Authorization, and Security 36
17.1 Authentication flow 36
17.2 authenticate middleware decision table 36
17.3 Verified security controls 36
17.4 Risks and potential vulnerabilities 37
17.5 Injection and data exposure 37
18. Error Handling and Logging 38
18.1 Error model 38
18.2 Logging 38
18.3 Health checks and monitoring 38
18.4 Retry and recovery 39
18.5 Debugging guide 39
18.6 Known gaps (verified) 39
19. Testing and Debugging 40
19.1 Current state 40
19.2 Static verification 40
19.3 Manual smoke test used for this document 40
19.4 Useful breakpoints 41
19.5 Troubleshooting 41
19.6 Suggested test strategy (recommendation) 41
20. Design Patterns and Code Quality 42
20.1 Strengths 42
20.2 Weaknesses and maintenance risks 42
21. Performance and Scalability Considerations 43
22. Known Limitations and Recommendations 44
23. End-to-End Execution Examples 45
Example A — Register, then log in 45
Example B — Duplicate registration 45
Example C — Malformed JSON 45
Example D — CORS 45
Example E — Caller-supplied correlation ID 45
24. Glossary 46
25. Appendix: Commands, Configuration, and File References 47
A. Command reference 47
B. Mermaid sources for the diagrams 47
C. Inspection checklist 49
D. Items that could not be verified 49

SECTION 2

Executive Summary

AgenticMeetingsManager is a TypeScript HTTP service built on Express 5. It is at an early stage. There are 16 TypeScript source files (572 lines, including comments and the fully commented-out Redis module). There are two public endpoints, no database, no tests and no deployment configuration.

2.1 What the backend does today

- Exposes `POST /auth/register` and `POST /auth/login`. Both are public; no route requires authentication.
- Uses a layered design, Router → `AuthController` → `AuthService` → `UserRepository`, wired by hand in `src/routes/auth.routes.ts`. There is no DI container.
- Stores users in a JavaScript array inside `UserRepository` (`src/repositories/user.repository.ts:15`). All data is lost on restart. `prisma/schema.prisma` is empty, Prisma is not a dependency, and the installed `pg` driver is never imported.
- Hashes passwords with bcrypt (cost factor 12). Login returns an HS256 JWT access token that expires after 15 minutes and is signed with `JWT_ACCESS_SECRET`.
- Contains a two-mode authentication middleware (`x-api-key` for services, Bearer JWT for users) in `src/middlewares/auth.middleware.ts`, but it is not attached to any route.
- Provides cross-cutting request-ID propagation, a CORS allowlist, structured pino logging, a central error handler, and graceful shutdown on SIGTERM/SIGINT.
- Ships the Redis client code fully commented out in the working tree. There are no queues, workers, scheduled jobs, outbound API integrations, tests, Dockerfile or CI configuration.

2.2 Most important findings

The following were confirmed by running the service locally on 26 October 2026. Details and evidence are in §17, §18 and §22.

#	Finding	Impact
1	No runtime input validation. A missing body or password returns 500, and registering without an email succeeds (201). zod is installed but unused.	Client mistakes surface as server errors; invalid records are stored.
2	Concurrent registrations with the same email all succeed. 3 of 3 parallel requests created 3 users.	Duplicate accounts. Check-then-insert race across await bcrypt.hash.
3	Malformed JSON returns 500 instead of 400, and the response has no requestId.	Misleading status; the failure cannot be correlated in logs.
4	The error middleware logs errors under the key error, which pino serialises as {}.	Error messages and stack traces are missing from logs.
5	Unknown routes return Express's default HTML 404. notFoundMiddleware exists but is not mounted.	Inconsistent, non-JSON error responses.
6	Login for an unknown email returns in ~0 ms; a wrong password takes ~165 ms. Register returns 409 for existing emails.	Account enumeration.
7	origin/master (merge 4da3b05) and origin/dev do not contain the register/login feature. The merge kept the older tree.	Branches disagree; see §3.3 before merging further.
8	dist/ is a stale build: it still connects to Redis at startup and has no CORS.	Running compiled output behaves differently from the source.

Table 1 — Highest-impact findings

2.3 How to read this document

Label	Meaning
VERIFIED	Read directly in the source; a repository-relative path and line number are given.
VERIFIED AT RUNTIME	Observed by starting the server locally and sending HTTP requests (see §19.3 for the exact script).
INFERENCE	Follows from the code or from documented library behaviour, but was not exercised.
RECOMMENDATION	A suggested improvement. Not existing functionality.
NOT APPLICABLE	The requested topic does not exist in this repository. The section says what evidence shows that.

Method. Every tracked file and the untracked `cors.middleware.ts` were read in full. Line numbers refer to the working tree on 26 October 2026. Runtime checks used `NODE_ENV=doccheck`, which points at a non-existent env file, so no real configuration was loaded, plus a test-only JWT secret. The contents of `envs/.env.dev` and `envs/.env.qa` were deliberately not opened because they are git-ignored and likely hold secrets. The list of environment variables is derived from the `process.env` reads in the code.

SECTION 3

Backend Project Overview

3.1 Project identity

Field	Details
Name / version	agentic-meetings-manager 1.0.0 (package.json:2–3)
Entry point	src/index.ts (package.json:4 main; all run scripts execute it with tsx)
Language	TypeScript (strict mode), compiled to CommonJS / ES2020
Framework	Express 5.2.1
Licence	ISC (package.json:13). The description field and README.md are empty.
Domain code	None yet. A search for "meeting" or "agent" in src/ only matches the logger's service name (src/config/logger.ts:9).

3.2 Version-control history

Commit	Date	Message	Branches
`4da3b05`	2026-08-29	Merge pull request #1 from shivaram9398/foundation	origin/master
`306d9c0`	2026-08-25	feat: implement authentication with user registration and login functionality	foundation (checked out), origin/foundation
`719ee6f`	2026-08-24	feat: add JWT authentication and logging middleware	origin/dev
`aefdc63`	2026-08-22	feat: initialize project with environment configuration and Redis setup	—
`71b4292`	2026-08-19	add tsconfig.json for TypeScript configuration	—
`c6a7bba`	2026-08-18	first commit	—

Table 2 — Full commit history (git log --all)

3.3 Branch state: the merged feature is missing from master

VERIFIED

`git diff 719ee6f origin/master` is empty. The merge commit 4da3b05 has both 719ee6f and 306d9c0 as parents, but its file tree equals 719ee6f. Compared with `foundation`, `origin/master` lacks the controller, service, repository, routes and `AppError`, and does not list `bcrypt` in package.json. Its `src/config/logger.ts` also has `process.env.NODE_ENV === "dev" || "qa"`, which is always truthy, so production would log at debug level with pino-pretty.

INFERENCE

Git records 306d9c0 as already merged into master, so merging `foundation` into master again will not restore the missing files. Recovering them on master needs a revert of the merge or a cherry-pick or re-apply of 306d9c0.

3.4 Working tree versus the last commit

This document describes the working tree, which is the code a developer gets when running `npm run dev` today.

File	State	Change
src/index.ts	modified	Adds corsMiddleware (L9, L15), removes a duplicate express.json(), and comments out connectRedis() and the Redis quit in shutdown.
src/config/redis.ts	modified	Entire file commented out. The committed version creates a client from REDIS_URL and exports connectRedis().
src/middlewares/cors.middleware.ts	untracked	New CORS allowlist middleware driven by CORS_ORIGINS.

INFERENCE

At commit 306d9c0, `startServer()` awaits `connectRedis()` before listening. If Redis is unreachable, the catch block logs `Failed to start server` and calls `process.exit(1)`. The committed code therefore needs a running Redis even though nothing uses it.

3.5 Capability matrix

Capability	Status	Evidence
User registration	Implemented	src/services/auth.service.ts:21–37
Login + JWT issuance	Implemented	src/services/auth.service.ts:39–73
Route protection	Written, not mounted	src/middlewares/auth.middleware.ts — no importer
Persistent database	Not implemented	Empty prisma/schema.prisma; pg never imported
Caching / Redis	Disabled	src/config/redis.ts commented out
Input validation	Not implemented	zod in package.json, no import
Background jobs / queues	Not implemented	No queue libraries or workers
Automated tests	Not implemented	No test files, framework or npm test script
Containerisation / CI	Not implemented	No Dockerfile, compose or pipeline files
Meetings / agent features	Not implemented	No related code

SECTION 4

Repository Structure

Directory tree (generated folders summarised)

```
Agentic-Meetings-Manager/
├── package.json              npm manifest: scripts and dependency ranges
├── package-lock.json         npm lockfile (lockfileVersion 3)
├── tsconfig.json             compiler options: src/ → dist/, strict, CommonJS
├── README.md                 empty (0 bytes)
├── .gitignore                ignores envs/.env.*, dist/, .vscode/, logs, coverage …
├── .vscode/settings.json     format-on-save with Prettier             [git-ignored]
├── envs/                     per-environment dotenv files              [git-ignored]
│   ├── .env.dev
│   └── .env.qa               (no .env.prod, no .env.example)
├── prisma/schema.prisma      empty (0 bytes), tracked
├── docs/                     empty directory
├── dist/                     stale tsc output with .js.map files      [git-ignored]
├── node_modules/             installed packages
└── src/
    ├── index.ts              bootstrap: middleware chain, listen, shutdown
    ├── config/
    │   ├── env.ts            loads envs/.env.<NODE_ENV> via dotenv
    │   ├── logger.ts         pino logger instance
    │   └── redis.ts          Redis client — fully commented out
    ├── middlewares/
    │   ├── request-id.middleware.ts   x-request-id propagation
    │   ├── cors.middleware.ts         CORS allowlist          [untracked]
    │   ├── logger.middleware.ts       access log per request
    │   ├── error.middleware.ts        central error → JSON
    │   ├── auth.middleware.ts         x-api-key / Bearer JWT  (unused)
    │   └── not-found.middleware.ts    JSON 404                (unused)
    ├── routes/auth.routes.ts          /auth router + object wiring
    ├── controllers/auth.controller.ts HTTP adapter for auth
    ├── services/auth.service.ts       registration / login rules
    ├── repositories/user.repository.ts in-memory user store
    ├── utils/app-error.ts             AppError(message, statusCode)
    └── types/express.d.ts             Request type augmentation
```

4.1 Directory responsibilities

Directory	Role	Notes
`src/config`	Process-wide singletons: environment loading, logger, (disabled) Redis client	Imported for side effects; env.ts must load first
`src/middlewares`	Express middleware functions, global and per-route	Two of six files are not used
`src/routes`	Express Routers and object wiring (composition root for each feature)	One router: auth
`src/controllers`	Translate HTTP ↔ service calls and choose status codes	No validation here
`src/services`	Business rules	Throws AppError for expected failures
`src/repositories`	Data access abstraction	In-memory implementation only
`src/utils`	Shared helpers	AppError only
`src/types`	Ambient TypeScript declarations	Compile-time only
`envs`	dotenv files selected by NODE_ENV	Not committed
`prisma`	Placeholder for a Prisma schema	Empty; Prisma not installed
`dist`	Output of npm run build	Out of date; not used by any script

4.2 What is tracked by git

Tracked: package.json, package-lock.json, tsconfig.json, README.md, .gitignore, prisma/schema.prisma and the 15 committed files under src/. Not tracked: `src/middlewares/cors.middleware.ts` (new), and the ignored paths `envs/`, `dist/`, `.vscode/` and `node_modules/`. The empty `docs/` directory is invisible to git.

SECTION 5

Complete File-by-File Reference

Each source file is listed in execution order: bootstrap first, then configuration, middleware, and the auth feature from router to repository. Package and tooling files are covered in §6 and §7.

src/index.ts (100 lines)

Field	Details
Responsibility	Process entry point and composition root. Builds the Express app, registers global middleware and routers in order, starts the HTTP server and installs shutdown handlers.
Key symbols	app (L11, not exported); startServer() (L20–98); inner shutdown(signal) (L38–75). No exports.
Imports	./config/env (side effect, L1), express, ./config/logger, ./routes/auth.routes, request-id, logger, error and cors middlewares
Used by	npm scripts dev / qa / prod (tsx src/index.ts); package.json main
Execution role	Runs once at start. Defines the middleware order for every request.

- L13–18 registration order: `express.json()` → `requestIdMiddleware` → `corsMiddleware` → `loggerMiddleware` → `/auth` router → `errorMiddleware`.
- L24 `app.listen(process.env.PORT, …)` passes the raw string. If PORT is unset, Node binds a random free port (§8).
- L22, L52–56: Redis connect and quit are commented out. L58–59 hold a `disconnectDatabase()` placeholder comment; no such function exists.
- The try/catch (L21, L88–97) covers only synchronous startup. No `server.on('error')` listener exists, so a listen failure such as EADDRINUSE is not handled by this code (INFERENCE).

src/config/env.ts (14 lines)

Field	Details
Responsibility	Selects and loads the dotenv file for the current environment.
Key symbols	environment = process.env.NODE_ENV || "dev" (L5); envPath (L7); dotenv.config (L8–10); export { environment } (L14)
Imports	dotenv, path, ./logger
Used by	src/index.ts:1 (side-effect import). The exported environment value is not imported anywhere.
Execution role	First module executed. Populates process.env before routes and CORS read it.

- Path is `<cwd>/envs/.env.<environment>`, relative to the working directory rather than the file, so the server must be started from the repository root.
- dotenv does not override variables that already exist in the real environment, and a missing file is not treated as an error. VERIFIED AT RUNTIME: with a non-existent `.env.doccheck` the server still started.
- Logs the environment name and absolute env-file path at info level (L11–12). dotenv 17 also prints its own banner, for example `injected env (0) from …`.

src/config/logger.ts (24 lines)

Field	Details
Responsibility	Creates the shared pino logger.
Key symbols	logger (L5–24). isDevelopment = NODE_ENV is "dev" or "qa" (L3).
Configuration	level debug (dev/qa) or info; base field service: "agentic-meetings-manager"; ISO timestamps; pino-pretty transport (colorised, pid/hostname hidden) only in dev/qa

Field	Details
Imports	pino (pino-pretty is loaded by name as a transport target)
Used by	config/env.ts, index.ts, cors / logger / error middlewares

- The logger is constructed when env.ts imports it, before `dotenv.config()` runs, so the log level and format depend on the NODE_ENV of the real process (set by `cross-env` in npm scripts), not on the env file.
- If NODE_ENV is unset, env.ts falls back to "dev" but the logger treats the process as non-development (info level, JSON).
- No `logger.debug` calls exist in the codebase, so the debug level currently has no effect.

src/config/redis.ts (21 lines, all commented)

Field	Details
Responsibility	(Disabled) Redis client creation and connect helper.
Committed version	createClient({ url: process.env.REDIS_URL }); 'error' event logged; connectRedis() awaits connect and logs success
Used by	Nothing in the working tree. index.ts:3 import is commented out.

src/types/express.d.ts (16 lines)

Field	Details
Responsibility	Global module augmentation of Express.Request
Adds	authType?: "user" | "service"; user?: { id?: string; email?: string }
Used by	auth.middleware.ts (type-checking of req.authType / req.user). Picked up through tsconfig include src//*.ts.

src/middlewares/request-id.middleware.ts (16 lines)

Field	Details
Responsibility	Ensures every request has a correlation ID.
Behaviour	Uses the incoming x-request-id header if present, otherwise crypto.randomUUID() (L9–10). Writes it back to req.headers (L12) and the response header (L13).
Imports	express types, node crypto
Used by	index.ts:14 (global, second in chain). Read later by logger and error middlewares through res.getHeader.

- The client-supplied value is accepted without length or format checks. VERIFIED AT RUNTIME: `x-request-id: trace-123` was echoed in the response and logs.

src/middlewares/cors.middleware.ts (30 lines, untracked)

Field	Details
Responsibility	CORS policy based on an allowlist
Behaviour	Allowlist parsed once from CORS_ORIGINS (comma-separated, trimmed, L7–10); warning logged if empty (L12–14). Requests without an Origin header are allowed. Unknown origins produce AppError(…, 403) (L24).
Options	credentials: true; methods GET, POST, PUT, PATCH, DELETE, OPTIONS; allowedHeaders Content-Type, Authorization, x-request-id; exposedHeaders x-request-id
Imports	cors, ../config/logger, ../utils/app-error
Used by	index.ts:15 (global, third)

- Rejection applies to any request that carries a non-allowlisted Origin, not only preflights. VERIFIED AT RUNTIME: POST with `Origin: http://evil.test` → 403 JSON.
- `x-api-key` is not in allowedHeaders, so browsers cannot send it cross-origin. This is acceptable if API keys are only used server-to-server (INFERENCE).

src/middlewares/logger.middleware.ts (27 lines)

Field	Details
Responsibility	One structured access-log line per completed response
Behaviour	Records start time (L9); on res 'finish' logs method, originalUrl, statusCode, duration "Nms" and requestId at info level (L11–24).
Used by	index.ts:16 (global, fourth). Requests rejected earlier (bad JSON, CORS) are never logged by it.

src/middlewares/error.middleware.ts (34 lines)

Field	Details
Responsibility	Final error handler (4-argument Express middleware)
Behaviour	Status is error.statusCode for AppError, else 500 (L14). Message is error.message for AppError, else "Internal server error" (L16–17). Logs at error level (L19–28) and responds { message, requestId } (L30–33).
Imports	../config/logger, ../utils/app-error
Used by	index.ts:18 (last). Receives errors from next(err), CORS rejections, JSON parse errors and synchronous throws in middleware.

- Non-AppError errors that carry an HTTP status, such as body-parser's 400 `entity.parse.failed`, become 500. VERIFIED AT RUNTIME.
- The error is logged under the key `error`, but pino's error serialiser only applies to `err`. The Error's message and stack are non-enumerable and are dropped: logs show `"error":{"statusCode":409}` or `{}`. VERIFIED AT RUNTIME.

src/middlewares/auth.middleware.ts (96 lines) — not mounted

Field	Details
Responsibility	Authenticate a request as either a service (x-api-key) or a user (Bearer JWT)
Key symbol	authenticate(req, res, next) (L6–96)
Imports	express types, jsonwebtoken, node crypto
Used by	Nothing (no import anywhere in src/)
Execution role	Intended per-route guard for future protected endpoints. Decision table in §17.2.

src/middlewares/not-found.middleware.ts (7 lines) — not mounted

Field	Details
Responsibility	JSON 404: { message: "Route <METHOD> <url> not found" }
Used by	Nothing. Unmatched routes fall through to Express's built-in HTML 404.

src/routes/auth.routes.ts (20 lines)

Field	Details
Responsibility	Declares the /auth endpoints and wires the auth feature's object graph
Key symbols	router (L6); userRepository (L9); authService (L11); authController (L13); POST /register (L16); POST /login (L18); default export

Field	Details
Imports	express Router, AuthController, AuthService, UserRepository
Used by	index.ts:17 mounted at /auth

- The objects are created once at module load, so a single `UserRepository` and its user array is shared by every request for the life of the process.

src/controllers/auth.controller.ts (41 lines)

Field	Details
Responsibility	HTTP adapter: passes req.body to the service, sets status codes and wraps results
Key symbols	class AuthController; register (L8–23) → 201; login (L25–40) → 200. Both are arrow-function properties so this survives being passed as a route handler.
Imports	express types, AuthService
Used by	auth.routes.ts

- Errors are caught and forwarded with `next(error)`. `req.body` is passed unchecked; the RegisterInput and LoginInput types are compile-time only.

src/services/auth.service.ts (82 lines)

Field	Details
Responsibility	Registration and login business rules
Key symbols	RegisterInput (L7–11); LoginInput (L13–16); class AuthService; register (L21–37); login (L39–73); private formatUser (L75–81)
Imports	bcrypt, jsonwebtoken, AppError, UserRepository and User type
Used by	AuthController

- register: duplicate check (409) → `bcrypt.hash(password, 12)` → `create` → user without password.
- login: lookup (401) → `bcrypt.compare` (401) → `jwt.sign({ sub, email }, JWT_ACCESS_SECRET, { expiresIn: "15m" })`.
- A missing secret throws a plain Error (L55), which the error middleware turns into 500.

src/repositories/user.repository.ts (33 lines)

Field	Details
Responsibility	User persistence abstraction (in-memory)
Key symbols	CreateUserInput (L1–5); User (L7–12); class UserRepository; private users: User[] (L15); findByEmail (L17–19); create (L21–32)
Imports	None. Uses the global crypto.randomUUID() (L23), available as a global in Node ≥ 19.
Used by	auth.routes.ts (instantiation), AuthService

- Both methods are `async` but do synchronous array work. The lookup is a linear `Array.find` with strict equality.

src/utils/app-error.ts (11 lines)

Field	Details
Responsibility	Error subclass carrying an HTTP status code for expected failures
Key symbols	class AppError(message, statusCode); Object.setPrototypeOf keeps instanceof reliable
Used by	auth.service.ts, cors.middleware.ts, error.middleware.ts

SECTION 6

Technology Stack and Dependencies

Technology	Installed	Kind	Status	Where used
Node.js	v24.15.0*	runtime	Active	Machine used for verification; no engines field or .nvmrc pins a version
TypeScript	7.0.2	dev	Active	tsconfig.json; npm run build; type-checking
tsx	4.23.12	dev	Active	Executes src/index.ts in every run script, including prod
cross-env	10.1.0	dev	Active	Sets NODE_ENV in npm scripts
Express	5.2.1	runtime	Active	src/index.ts, routes, middleware types
cors	2.8.6	runtime	Active (working tree)	src/middlewares/cors.middleware.ts
dotenv	17.4.2	runtime	Active	src/config/env.ts
pino	10.3.1	runtime	Active	src/config/logger.ts and all logging
pino-pretty	13.1.3	runtime	Active in dev/qa	Transport target in logger.ts
bcrypt	6.0.0	runtime	Active	AuthService hash / compare
jsonwebtoken	9.0.3	runtime	Active	AuthService.sign; auth.middleware verify (unused)
node:crypto	built-in	runtime	Active	randomUUID (request-id, repository), timingSafeEqual (auth middleware)
redis	6.2.1	runtime	Installed, unused	Only in commented-out src/config/redis.ts
pg	8.23.0	runtime	Installed, unused	Never imported
zod	4.4.3	runtime	Installed, unused	Never imported
prettier	3.9.6	dev	Editor only	.vscode/settings.json format-on-save; no npm script
@types/*	various	dev	Active	bcrypt, cors, express 5.0.6, jsonwebtoken, node 26.2.0
Prisma	—	—	Not installed	prisma/schema.prisma exists but is empty

Table 3 — Technologies discovered (versions read from node_modules/*/package.json). *Node version is the verification machine's, not a project setting.

6.1 Role of each active technology

Express 5

The HTTP framework. `express()` creates the app (index.ts:11), `Router()` groups the auth routes (auth.routes.ts:6), and `express.json()` parses JSON bodies with its default 100 kB limit. Express 5 forwards rejected promises from async handlers to the error middleware. The controllers use explicit try/catch anyway. Every module that receives HTTP traffic depends on it.

TypeScript + tsx

All source is TypeScript under `strict`. During development and in the `prod` script, `tsx` compiles and runs files on the fly without type-checking. `npm run build` (tsc) emits to `dist/`, but no script runs the built output. Type safety therefore depends on developers or editors running `tsc`. VERIFIED: `npx tsc --noEmit` exits 0.

dotenv + cross-env

`cross-env` sets NODE_ENV portably in npm scripts. `dotenv` then loads `envs/.env.<NODE_ENV>`. This is the only configuration mechanism; there is no schema validation.

pino + pino-pretty

Structured JSON logging with a fixed `service` field and ISO timestamps. In dev/qa the pino-pretty transport (which pino runs in a worker thread) gives coloured, human-readable output. Used for lifecycle events, access logs, CORS warnings and errors.

bcrypt

Native password hashing. `hash(password, 12)` at registration and `compare` at login. The native addon runs in libuv's thread pool, so hashing does not block the event loop. Measured locally: about 165 ms per operation at cost 12.

jsonwebtoken

Signs access tokens at login (HS256 by default with a string secret, 15-minute expiry). Also used for `jwt.verify` in the unmounted authenticate middleware.

cors

Implements the CORS response headers and the origin decision through a callback, driven by `CORS_ORIGINS`.

6.2 Not present

No ORM or query builder in use, no test framework, no API documentation generator (OpenAPI/Swagger), no security-header middleware (helmet), no rate limiter, no message broker or job queue, no APM or metrics library, no Dockerfile or CI configuration, and no linter.

SECTION 7

Backend Setup and Configuration

7.1 Prerequisites

- Node.js with global `crypto` (≥ 19, used without import in user.repository.ts). Verified on v24.15.0. INFERENCE: Express 5 also requires Node ≥ 18.
- npm (package-lock.json v3).
- `bcrypt` is a native module. Installation needs a prebuilt binary for the platform or a C++ build toolchain (INFERENCE).
- An env file at `envs/.env.<env>` or equivalent real environment variables (§8). Redis and PostgreSQL are not needed by the working tree.

7.2 Install and run

Commands

```
# from the repository root (env.ts resolves envs/ against the current directory)
npm install

npm run dev     # NODE_ENV=dev  · tsx watch · reload on change · pretty debug logs
npm run qa      # NODE_ENV=qa   · tsx watch · pretty debug logs
npm run prod    # NODE_ENV=prod · tsx (no watch) · JSON info logs · needs real env vars (no .env.prod)
npm run build   # tsc → dist/   (nothing runs dist/ afterwards)

npx tsc --noEmit   # type-check only (exit 0 on 2026-10-26)
```

Script	Command (package.json:6–9)	Notes
`dev`	`cross-env NODE_ENV=dev tsx watch src/index.ts`	Restarts on file change; loads envs/.env.dev
`qa`	`cross-env NODE_ENV=qa tsx watch src/index.ts`	Watch mode even for QA; loads envs/.env.qa
`prod`	`cross-env NODE_ENV=prod tsx src/index.ts`	Runs TypeScript through tsx at runtime; envs/.env.prod does not exist
`build`	`tsc`	Emits dist/ with source maps. No start script uses it.

There are no `start`, `test`, `lint` or `format` scripts.

7.3 TypeScript compiler options

Option	Value	Effect
`target / module`	`ES2020 / commonjs`	Output is CommonJS; imports become ordered require() calls
`rootDir / outDir`	`./src / ./dist`	Mirror layout in dist/
`strict`	`true`	All strict checks (null checks, noImplicitAny …)
`esModuleInterop`	`true`	Allows default imports of CommonJS packages (express, bcrypt …)
`noUnusedLocals / Parameters`	`true`	Unused names are compile errors (hence _next in error.middleware)
`noImplicitReturns`	`true`	Every code path must return
`sourceMap`	`true`	.js.map files for debugging compiled output
`include / exclude`	`src/**/*.ts / node_modules, dist`	Also picks up types/express.d.ts

7.4 How configuration is loaded

1. The npm script sets NODE_ENV in the real process environment (cross-env).
2. `src/index.ts:1` imports `./config/env`, which first imports `./config/logger`. The logger reads NODE_ENV and fixes its level and format.
3. `env.ts` computes `envs/.env.<NODE_ENV or "dev">` against `process.cwd()` and calls `dotenv.config()`. Existing variables are not overwritten, and a missing file is silently ignored.
4. `auth.routes.ts` and `cors.middleware.ts` are evaluated next. CORS reads `CORS_ORIGINS` once here.
5. `PORT` is read when `app.listen` runs. `JWT_ACCESS_SECRET` and `INTERNAL_API_KEY` are read on every request that needs them.

7.5 Middleware registration and execution order

#	Middleware	Registered	Purpose	On failure
1	`express.json()`	`index.ts:13`	Parse JSON body into req.body	Parse error → errorMiddleware → 500 (no requestId)
2	`requestIdMiddleware`	`index.ts:14`	Correlation ID	—
3	`corsMiddleware`	`index.ts:15`	Origin allowlist + CORS headers	AppError 403 → errorMiddleware
4	`loggerMiddleware`	`index.ts:16`	Access log on finish	—
5	`authRoutes`	`index.ts:17 (/auth)`	Feature endpoints	next(err) → errorMiddleware
6	`errorMiddleware`	`index.ts:18`	Uniform error JSON	—

Table 4 — Global middleware chain

7.6 Dependency wiring

There is no DI container. `auth.routes.ts` acts as the composition root: it constructs `UserRepository`, injects it into `AuthService` through the constructor, and injects the service into `AuthController`. Constructor parameter properties (`private readonly`) store the dependencies. The service depends on the concrete `UserRepository` class rather than an interface, so swapping the in-memory store for a database means editing this file and the type import in the service.

7.7 Deployment and containerisation

> **NOT APPLICABLE**
> No Dockerfile, docker-compose file, Procfile, CI pipeline, infrastructure-as-code or process-manager configuration exists. `npm run prod` is the only production-oriented entry point.

7.8 Configuration reference

Configuration / File	Purpose	Used by	Required settings
`package.json`	Scripts, dependency ranges, entry	npm, tsx	Scripts set NODE_ENV
`package-lock.json`	Exact dependency tree (v3)	npm install / ci	—
`tsconfig.json`	Compiler options	tsc, editors	rootDir src, outDir dist
`envs/.env.<env>`	Per-environment variables	src/config/env.ts	PORT, JWT_ACCESS_SECRET; CORS_ORIGINS for browsers
`src/config/env.ts`	Loads the env file	index.ts (first import)	NODE_ENV (defaults to dev)

Configuration / File	Purpose	Used by	Required settings
src/config/logger.ts	Logger level / format	All logging modules	NODE_ENV dev|qa for pretty debug output
src/config/redis.ts	Redis client (disabled)	nothing	REDIS_URL if re-enabled
src/middlewares/cors.middleware.ts	CORS policy	index.ts	CORS_ORIGINS (comma-separated)
.gitignore	Keeps env files, dist, IDE files out of git	git	Allows envs/.env.example (none exists)
.vscode/settings.json	Prettier format on save	VS Code	Prettier extension
prisma/schema.prisma	Placeholder	nothing	—

Table 5 — Configuration reference

SECTION 8

Environment Variables

> **NOTE**
> Values are intentionally not reproduced. `envs/.env.dev` and `envs/.env.qa` exist but were not opened. The variables below are every `process.env` read in `src/`.

Variable	Purpose	Read at	When	Required?	If missing
`NODE_ENV`	Selects env file and log mode	`config/env.ts:5` `config/logger.ts:3` `index.ts:28`	start-up	Set by scripts	env.ts uses "dev" but the logger switches to JSON/ info: the two disagree
`PORT`	HTTP listen port	`index.ts:24, 27`	start-up	Effectively yes	Server starts on a random OS-assigned port; the 'Server started' log has no port field (VERIFIED AT RUNTIME)
`JWT_ACCESS_SECRET`	HMAC key to sign/ verify access tokens	`services/auth.service.ts:52` `middlewares/auth.middleware.ts:62`	per request	Yes, for login	A login with valid credentials throws → 500. Not checked at start-up.
`CORS_ORIGINS`	Comma-separated browser origin allowlist	`middlewares/cors.middleware.ts:7`	module load	Optional	Warning logged; every request with an Origin header gets 403 (VERIFIED AT RUNTIME)
`INTERNAL_API_KEY`	Shared key for service callers	`middlewares/auth.middleware.ts:31`	per request	Only if authenticate is used	Throws → 500 when an x-api-key header is sent
`REDIS_URL`	Redis connection URL	`config/redis.ts:5 (commented)`	—	No (disabled)	No effect in the working tree

Table 6 — Environment variables referenced by the code

8.1 Environment files

- Present: `envs/.env.dev`, `envs/.env.qa`. Absent: `envs/.env.prod`, so `npm run prod` relies entirely on real environment variables. There is no `envs/.env.example`, although `.gitignore` explicitly allows one.
- No validation: wrong or missing values are only discovered when they are used (§22, recommendation R6).

> **RECOMMENDATION**
> Add `envs/.env.example` listing the variable names above with placeholder values, and validate them at start-up with a zod schema in `src/config/env.ts`. zod is already a dependency.

SECTION 9

High-Level Architecture

The service is a single Node.js process running one Express application. A request passes through four global middleware, reaches a feature router, and is handled by a controller → service → repository chain. All state lives in process memory. The only real I/O besides inbound HTTP is reading the env file at start-up and writing logs to stdout.

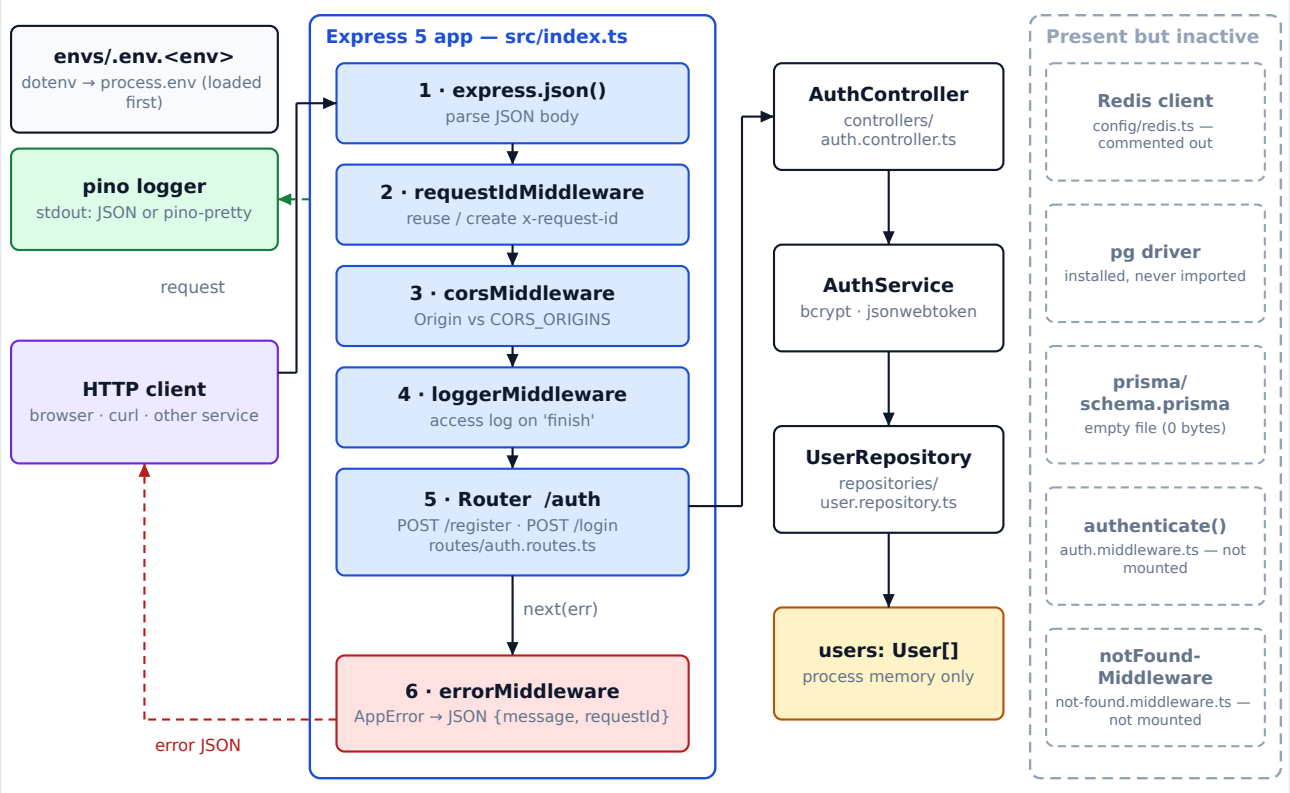


Figure 1 — High-level backend architecture (working tree). Solid boxes are executed at runtime; dashed boxes exist in the repository but are never executed. Success responses are written directly by AuthController.

How to read Figure 1. The left column holds the process's inputs and outputs: configuration, log output and callers. The blue middle frame is the Express pipeline, numbered in registration order. The right-hand chain is the auth feature. The dashed column lists code that exists but never runs; each item is a starting point for future work (Redis, a database, protected routes, JSON 404s).

9.1 Layers

Layer	Files	Responsibility	Depends on
Bootstrap	`src/index.ts`	Assemble app, listen, shut down	config, middlewares, routes
Configuration	`src/config/*.ts`	Env loading, logger	dotenv, pino
Middleware	`src/middlewares/*.ts`	Cross-cutting HTTP concerns	logger, AppError
Routing	`src/routes/auth.routes.ts`	URL → handler; object wiring	controller, service, repository
Controller	`src/controllers/auth.controller.ts`	HTTP in/out, status codes	service
Service	`src/services/auth.service.ts`	Business rules, hashing, tokens	repository, bcrypt, jsonwebtoken

Layer	Files	Responsibility	Depends on
Repository	`src/repositories/ user.repository.ts`	Data storage (in-memory)	—
Shared	`src/utils, src/types`	Error type, type augmentation	—

9.2 Module dependency graph

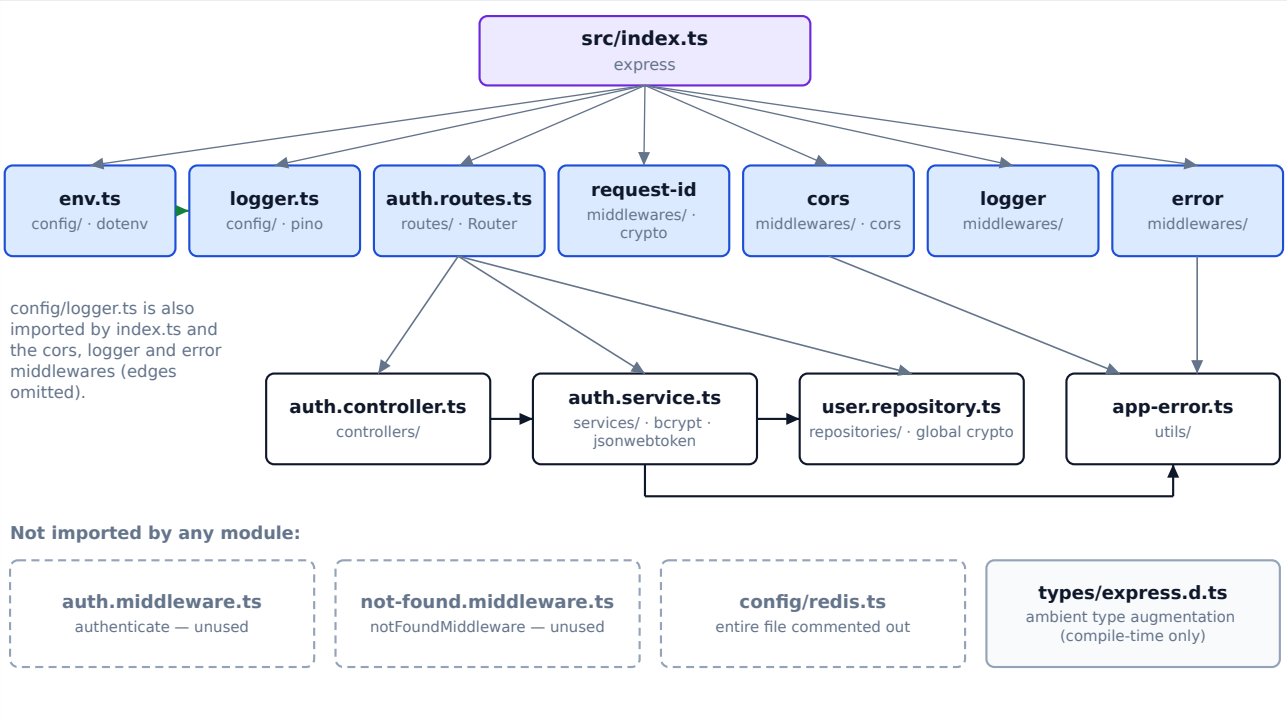


Figure 2 — Module dependency graph from import statements. Arrows point from importer to imported module; package names show the main third-party dependency of each file.

Dependencies point one way: routes → controller → service → repository. Nothing lower in the stack imports anything higher. `AppError` is the only module shared between the business layer and the HTTP layer. The service throws it and the error middleware maps it to a status code.

SECTION 10

Application Startup and Lifecycle

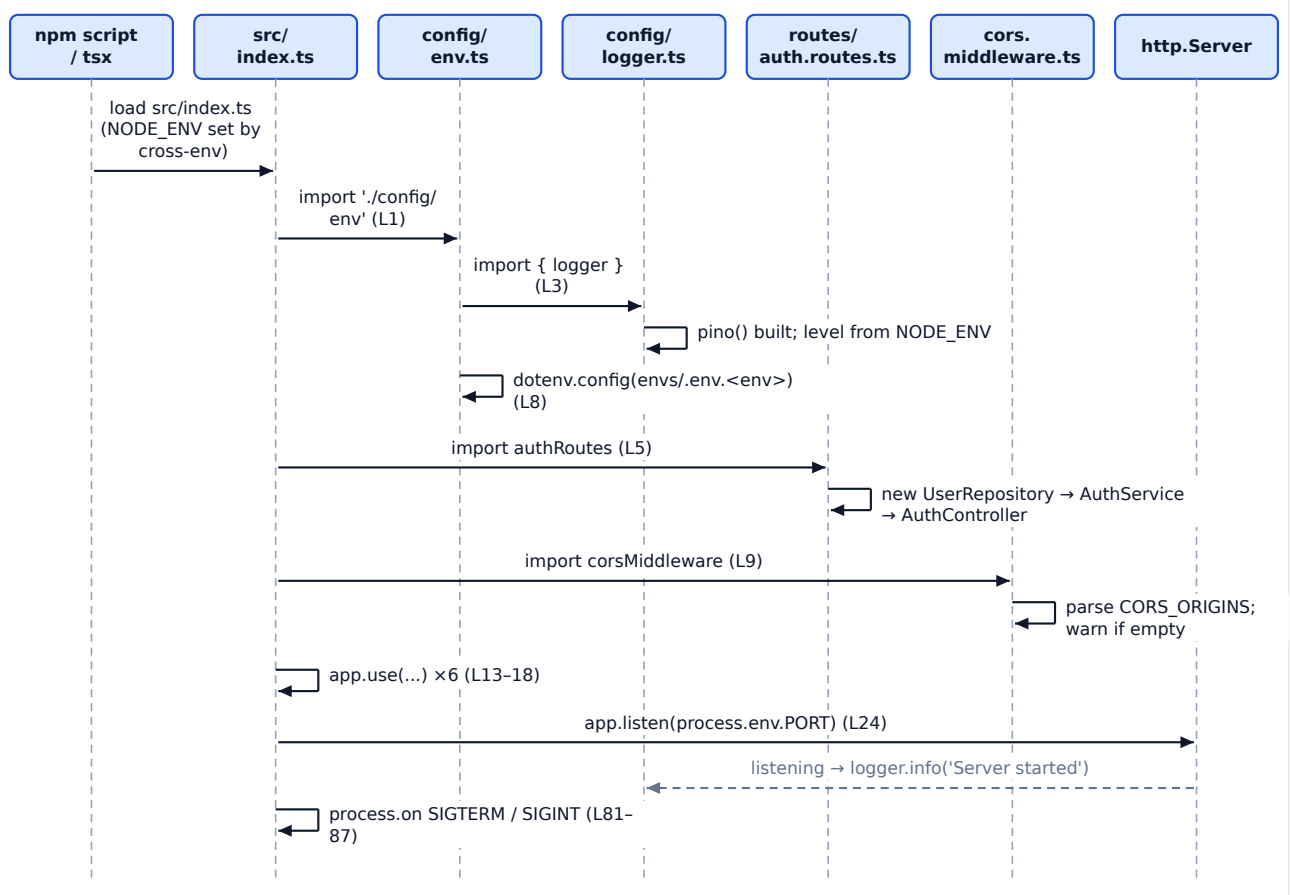


Figure 3 — Application startup sequence. CommonJS executes imports in source order, so the logger is built before dotenv runs and the route module wires its objects before CORS is configured.

10.1 Startup step by step

1. `npm run dev` → `cross-env NODE_ENV=dev tsx watch src/index.ts`.
2. `src/index.ts` is compiled to CommonJS by tsx, so its imports run in source order.
3. `config/env.ts` → `config/logger.ts`: pino is built with NODE_ENV from the real environment.
4. `env.ts` calls `dotenv.config({ path: envs/.env.dev })` and logs `Environment: dev` and the file path.
5. `express`, then `routes/auth.routes.ts`: this pulls in the controller, service (bcrypt, jsonwebtoken), repository and AppError, and creates one instance of each.
6. Logger, request-id and error middleware modules load (no side effects). `cors.middleware.ts` loads and parses CORS_ORIGINS, warning if empty.
7. `const app = express()` and six `app.use` calls (L11–18).
8. `startServer()` runs. `app.listen(process.env.PORT)` binds the port, and the callback logs `Server started` with port and environment.
9. SIGTERM and SIGINT handlers are registered. The process now idles on the event loop.

> **VERIFIED AT RUNTIME**
> Observed log order with a throwaway config: `Environment: doccheck` → `Loaded env file: …/envs/.env.doccheck` → `CORS_ORIGINS is not set …` (warn) → `Server started`.

10.2 Ordering subtleties

- **Logger before dotenv.** Setting NODE_ENV inside an env file has no effect on logging. Only the real environment counts.
- **Configuration read at import time.** CORS_ORIGINS is captured once. Changing it requires a restart (tsx watch restarts only on source changes).
- **Singletons at import time.** The user store is created when `auth.routes.ts` is first required, before the server listens.
- **Unhandled listen errors.** There is no `server.on('error')`. INFERENCE: a port conflict raises an unhandled 'error' event and the process crashes with a stack trace instead of the `Failed to start server` log.

10.3 Graceful shutdown

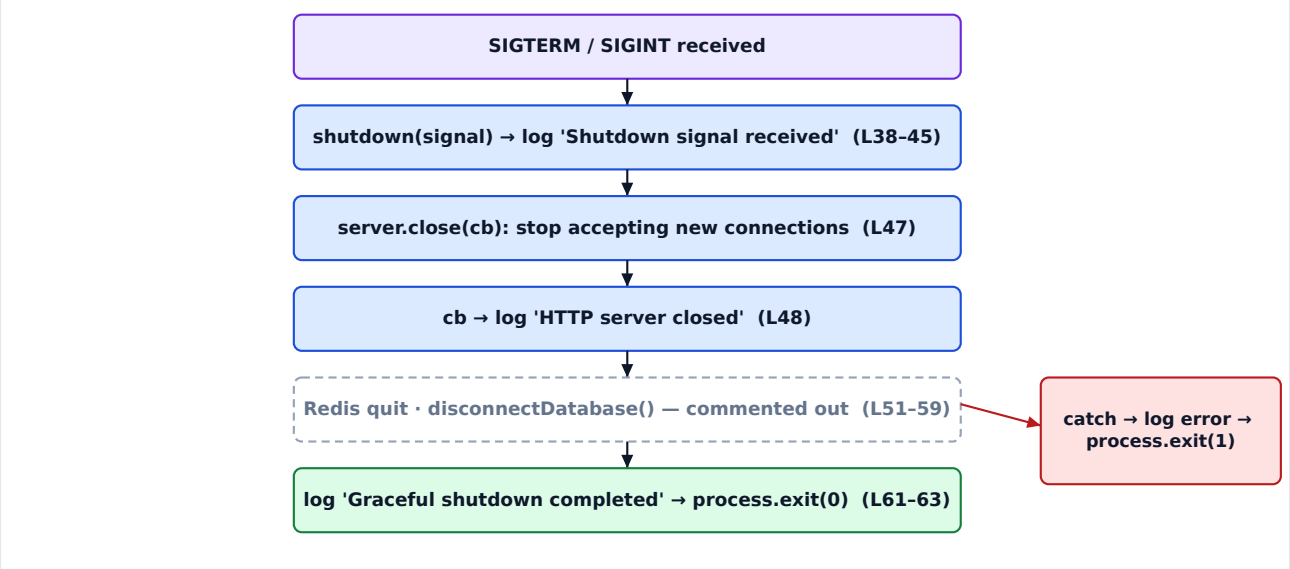


Figure 4 — Graceful shutdown flow in src/index.ts (sequence confirmed at runtime by sending SIGTERM).

On SIGTERM or SIGINT, `shutdown()` logs the signal and calls `server.close()`. This stops new connections, and the callback runs after existing connections end. The callback logs, skips the commented-out Redis and database cleanup, logs completion and exits with code 0. There is no forced-exit timeout, so a request that never finishes would delay exit indefinitely (INFERENCE).

> **VERIFIED AT RUNTIME**
> Sending SIGTERM produced: `Shutdown signal received` (signal SIGTERM) → `HTTP server closed` → `Graceful shutdown completed`.

SECTION 11

API Endpoint Reference

All routes come from `src/routes/auth.routes.ts`, mounted at `/auth` in `src/index.ts:17`. There is no global prefix (such as /api) and no version segment. The base URL is `http://<host>:<PORT>`.

Method	Path	Auth	Handler → Service	Success	Errors
POST	/auth/register	Public	AuthController.register → AuthService.register	201	409, 500, 403 (CORS)
POST	/auth/login	Public	AuthController.login → AuthService.login	200	401, 500, 403 (CORS)

Table 7 — Complete endpoint inventory

11.1 POST /auth/register

Field	Details
HTTP method	POST
Route	/auth/register (auth.routes.ts:16)
Purpose	Create a user account in the in-memory store
Handler	AuthController.register (auth.controller.ts:8–23)
Service	AuthService.register (auth.service.ts:21–37) → UserRepository.findByEmail, UserRepository.create
Authentication	None (public). Global middleware applies: JSON parsing, request ID, CORS, access log.
Input	Header Content-Type: application/json. Optional x-request-id. Body (TypeScript RegisterInput, compile-time only): { name: string, email: string, password: string }
Validation	None at runtime. Observed: no body → 500; missing password → 500 (bcrypt rejects undefined); missing email → 201 with the user stored without an email; email comparison is case-sensitive.
Output	201 { message: "User registered successfully", data: { id, name, email } }. The password hash is never returned.
Side effects	Appends a User to UserRepository.users; one bcrypt hash at cost 12 (~165 ms on the libuv thread pool); one access-log line
Errors	409 { message: "User already exists", requestId } · 500 { message: "Internal server error", requestId } · 403 disallowed Origin · 500 malformed JSON (no requestId)

Example captured at runtime (test data)

```
POST /auth/register
Content-Type: application/json

{"name":"Ann","email":"a@x.io","password":"<redacted>"}

HTTP/1.1 201 Created
x-request-id: 5683953d-9233-4011-acd8-f302badf52dd
{"message":"User registered successfully",
 "data":{"id":"2a038b70-3e6c-469a-b71c-4d391cf439e2","name":"Ann","email":"a@x.io"}}
```

11.2 POST /auth/login

Field	Details
HTTP method	POST

Field	Details
Route	/auth/login (auth.routes.ts:18)
Purpose	Verify credentials and issue a short-lived access token
Handler	AuthController.login (auth.controller.ts:25–40)
Service	AuthService.login (auth.service.ts:39–73) → UserRepository.findByEmail, bcrypt.compare, jwt.sign
Authentication	None (public)
Input	Body (LoginInput, compile-time only): { email: string, password: string }
Validation	None at runtime. An empty object {} returns 401 (no user has an undefined email unless one was registered without an email; see §13.3).
Output	200 { message: "Login successful", data: { accessToken, user: { id, name, email } } }
Token	HS256 JWT; claims sub (user id), email, iat, exp; lifetime 900 s; secret JWT_ACCESS_SECRET. No refresh token is issued.
Side effects	One bcrypt compare (only if the email exists); one access-log line. Nothing is persisted.
Errors	401 { message: "Invalid email or password", requestId } for unknown email or wrong password · 500 if JWT_ACCESS_SECRET is unset or the body is missing or malformed · 403 disallowed Origin

Example captured at runtime (token redacted)

```
POST /auth/login
Content-Type: application/json

{"email":"a@x.io","password":"<redacted>"}

HTTP/1.1 200 OK
{"message":"Login successful",
 "data":{"accessToken":"<JWT redacted>",
         "user":{"id":"2a038b70-3e6c-469a-b71c-4d391cf439e2","name":"Ann","email":"a@x.io"}}}

decoded JWT header : {"alg":"HS256","typ":"JWT"}
decoded claim keys : sub, email, iat, exp        (exp - iat = 900)
```

11.3 Cross-cutting API behaviour

Aspect	Behaviour
Success envelope	{ message: string, data: object } (controller)
Error envelope	{ message: string, requestId: string } (errorMiddleware). Express's default 404 is HTML, not JSON.
Response headers	x-request-id on every request that reaches middleware #2; X-Powered-By: Express (Express default, not disabled); ETag; CORS headers (Vary: Origin, Access-Control-Allow-Credentials, Access-Control-Expose-Headers)
Unknown routes / methods	GET /nope → 404 HTML "Cannot GET /nope" (VERIFIED AT RUNTIME). The same applies to GET /auth/login.
Preflight	OPTIONS from an allowlisted origin → 204 with Allow-Origin, Allow-Credentials, Allow-Methods, Allow-Headers (VERIFIED AT RUNTIME)
Body limit	express.json() default (100 kB). INFERENCE: oversize bodies produce a 413 error that the error middleware turns into 500.
Pagination / filtering / sorting / search	Not applicable: no list endpoints

Aspect	Behaviour
Versioning	None
Rate limiting	None
Protected endpoints	None. authenticate exists but is unused.

SECTION 12

Controller, Service, and Repository Execution Flows

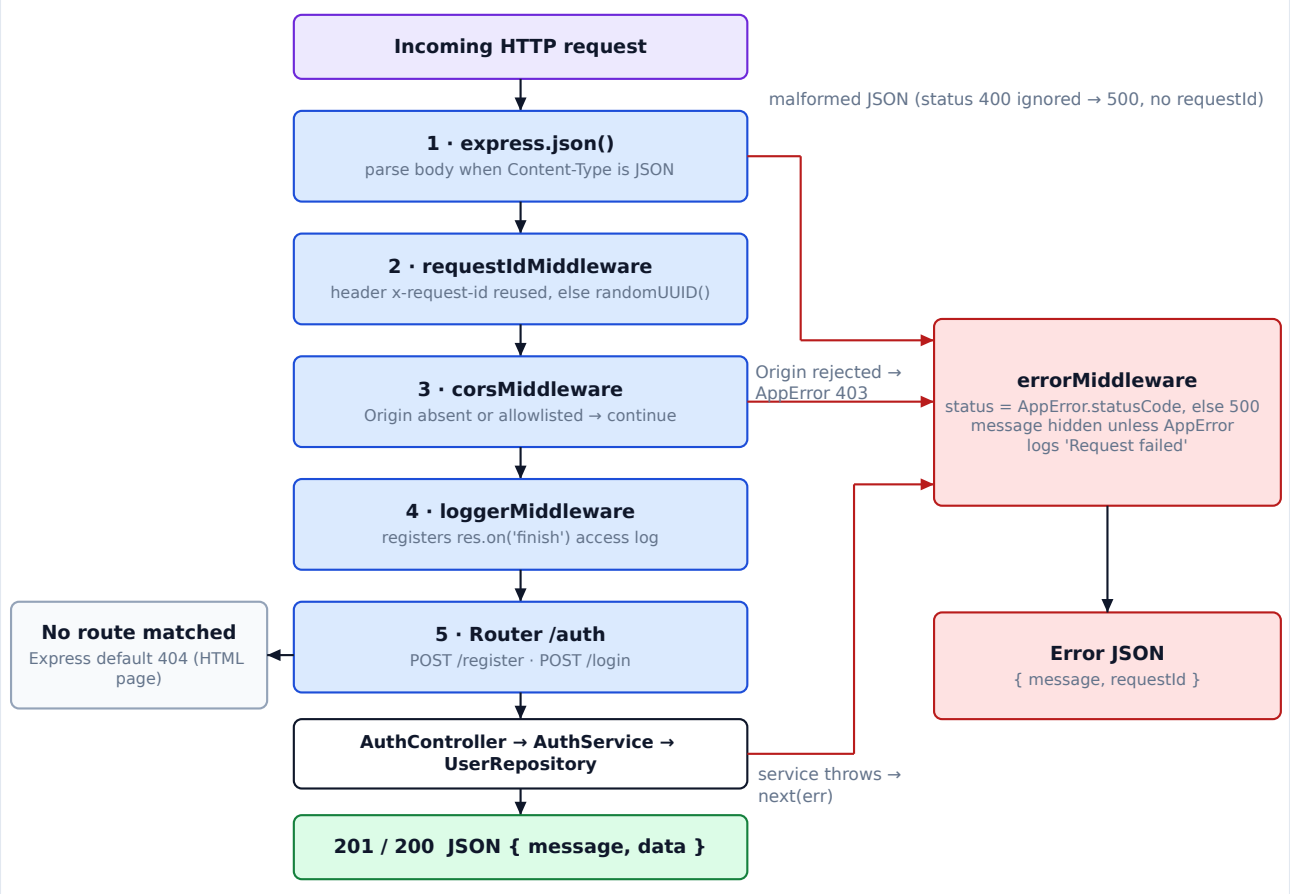


Figure 5 — API request lifecycle (verified by code reading and runtime checks). Errors raised before step 2 carry no request ID, and requests that fail before step 4 produce no access-log line.

12.1 Following a request through the code

1. Node's HTTP server hands the request to the Express app built in `src/index.ts:11`.
2. `express.json()` (L13) fills `req.body` when Content-Type is JSON. With no body, `req.body` stays undefined in Express 5.
3. `requestIdMiddleware` (request-id.middleware.ts:9–13) settles the correlation ID and sets the response header.
4. `corsMiddleware` (cors.middleware.ts:17–25) checks Origin. If allowed, it adds CORS headers; if not, it calls `next(AppError 403)`.
5. `loggerMiddleware` (logger.middleware.ts:9–24) records the start time and subscribes to `finish`.
6. The `/auth` router (auth.routes.ts:16 or 18) calls the bound controller method.
7. The controller awaits the service. On success it writes JSON with 201 or 200. On a throw it calls `next(error)`.
8. On error, `errorMiddleware` (error.middleware.ts:6–34) logs `Request failed` and writes `{ message, requestId }`.
9. When the response finishes, the access log `HTTP request completed` is written with status and duration.

12.2 Registration flow

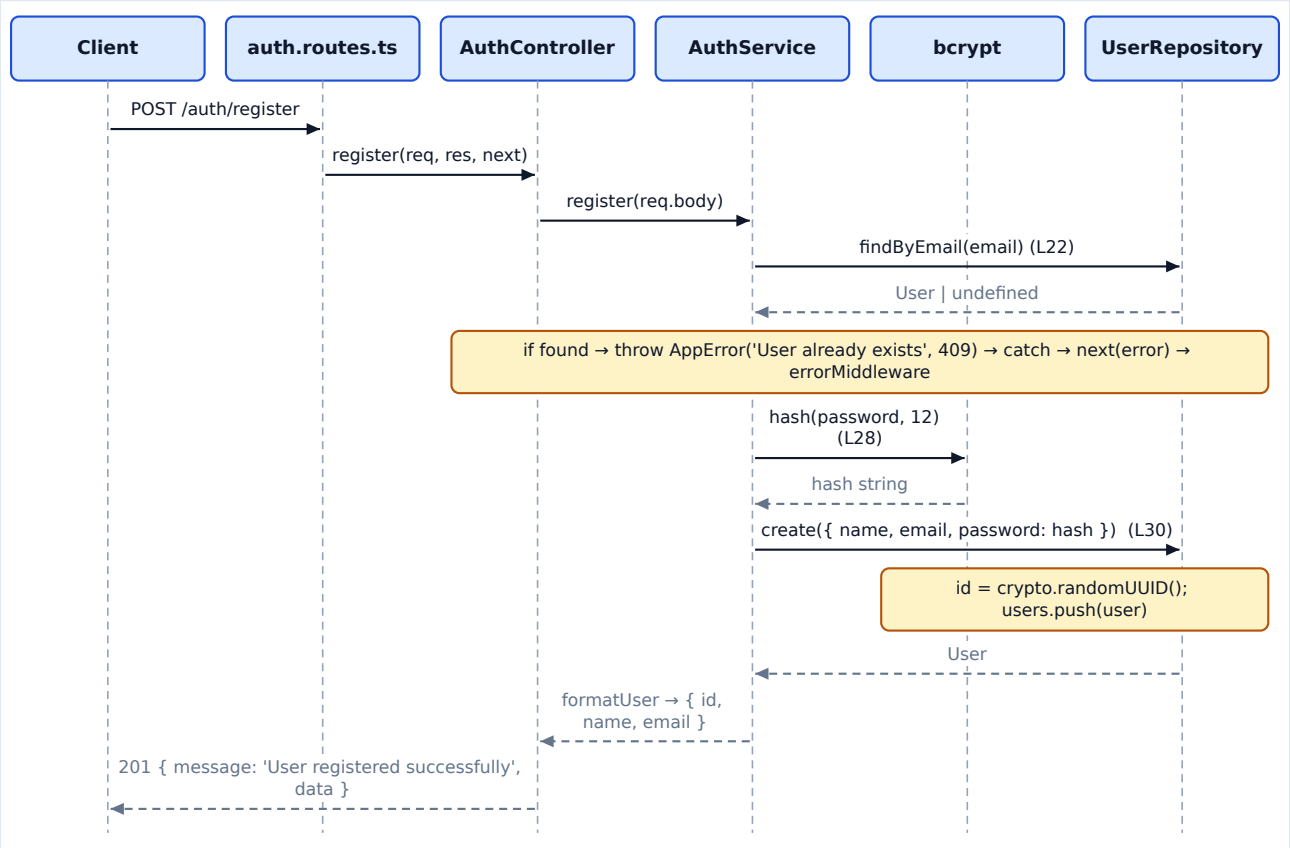


Figure 6 — POST /auth/register execution sequence (src/services/auth.service.ts:21–37).

12.3 Login flow

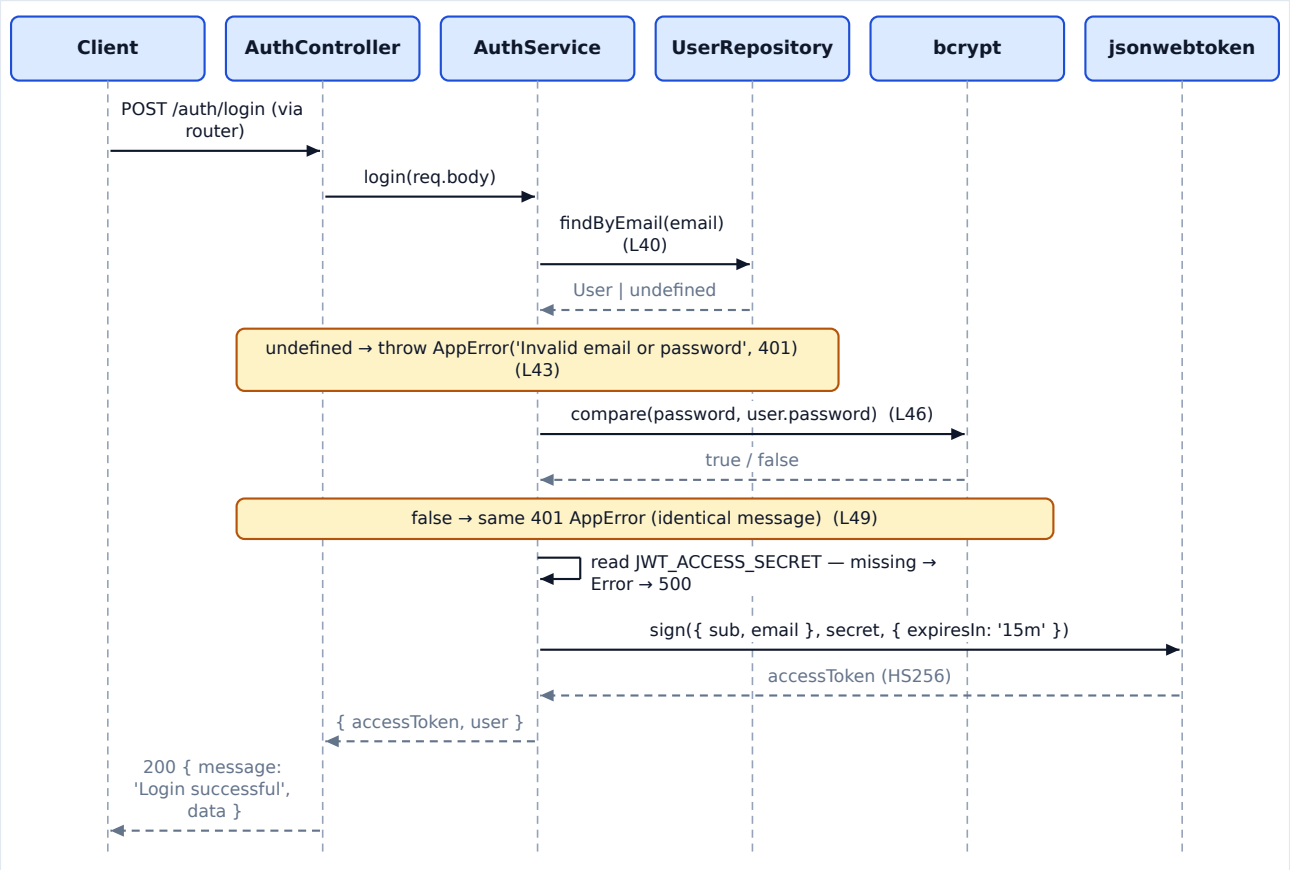


Figure 7 — POST /auth/login execution sequence (src/services/auth.service.ts:39–73).

12.4 Error propagation

Expected business failures are `AppError` instances thrown in `AuthService` and propagated with `next(error)` by the controller's catch block. Unexpected failures, such as a bcrypt rejection on undefined input, a TypeError from a missing body, or a missing secret, take the same path but are reported as a generic 500. Middleware failures (JSON parsing, CORS) go straight to `errorMiddleware` because Express skips the remaining non-error middleware. No layer retries, wraps or translates errors.

SECTION 13

Database Architecture and Data Flow

> **NOT APPLICABLE**
> There is no database. No connection is opened, no schema is defined (`prisma/schema.prisma` is 0 bytes), and there are no migrations or seed files. The only data store is an array inside `UserRepository`. Everything below describes that store and the dormant persistence pieces.

13.1 The actual data store

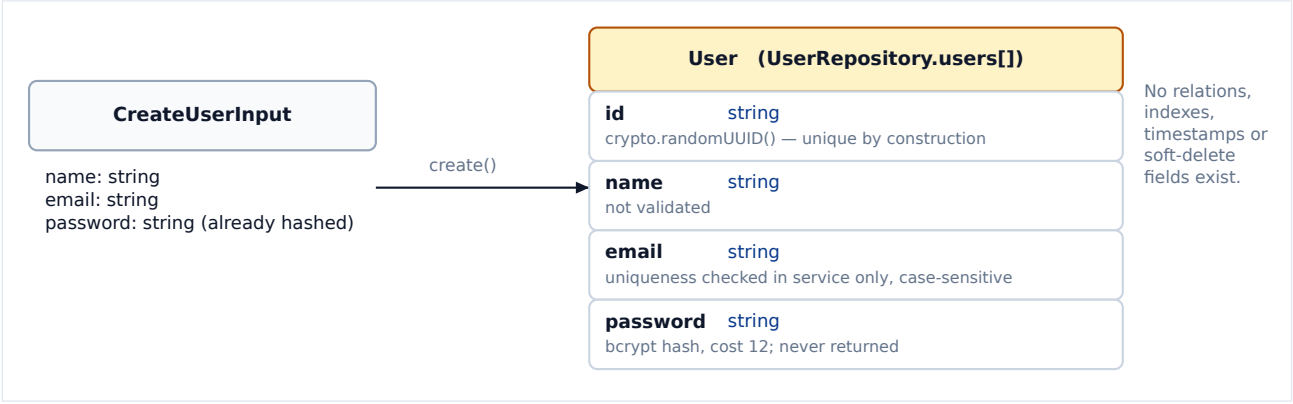


Figure 8 — The only data entity: User, held in a process-local array. No database schema exists (prisma/schema.prisma is empty).

Field	Type	Set by	Constraint / meaning
`id`	`string`	UserRepository.create — crypto.randomUUID() (L23)	Primary identifier; becomes the JWT sub claim
`name`	`string`	Request body via AuthService.register	Display name; never validated
`email`	`string`	Request body	Login key. Uniqueness is checked only in AuthService.register (L22–26), by exact case-sensitive match.
`password`	`string`	bcrypt.hash(password, 12) in AuthService (L28)	Hash only; removed by formatUser before any response

Table 8 — User record (src/repositories/user.repository.ts:7–12)

13.2 Operations

Operation	Method	Implementation
Create	`create(input: CreateUserInput)`	Builds a User with a new UUID, pushes it onto users, returns it
Read	`findByEmail(email)`	users.find(u => u.email === email): linear scan, first match
Update / Delete / List	–	Not implemented

13.3 Consistency, transactions and concurrency

- **No transactions.** Each method is a single synchronous array operation, so individual calls are atomic within the JavaScript event loop.
- **Check-then-insert race.** `register` awaits `findByEmail`, then awaits `bcrypt.hash` (thread pool, roughly 165 ms), then calls `create`. Concurrent requests for the same email all pass the check before any of them inserts. VERIFIED AT RUNTIME: 3 parallel registrations of `race@x.io` returned 201 three

times with three different IDs.

- **Missing values compare equal.** A user registered without an email has `email: undefined`, and a second such registration is rejected with 409 because `undefined === undefined`. VERIFIED AT RUNTIME.
- **Case sensitivity.** `RACE@x.io` and `race@x.io` are different users. VERIFIED AT RUNTIME.

13.4 Lifecycle and scope of the data

- Created when `auth.routes.ts` is first imported. Destroyed when the process exits, including every `tsx watch` restart during development.
- Not shared between processes. Two instances behind a load balancer would each hold their own users. A token issued by one instance would still verify on another with the same secret, but the user might not exist there (INFERENCE).
- Grows without bound; there is no eviction or deletion.

13.5 Dormant persistence artefacts

Artefact	State
`pg@8.23.0`	Declared dependency; not imported anywhere
`prisma/schema.prisma`	Tracked, empty. Neither prisma nor @prisma/client is installed.
`src/index.ts:58–59`	Comment placeholder // await disconnectDatabase();; no such function exists
`Migrations / seeds`	None

INFERENCE
The dependencies suggest PostgreSQL is the intended database, possibly through Prisma. The code gives no evidence of the planned schema, so no tables or relationships are documented here.

RECOMMENDATION
When persistence is added, keep the `UserRepository` method signatures, enforce a unique index on a normalised (lower-cased) email column, and handle the unique-violation error as 409. That fixes the race in §13.3 at the storage layer.

SECTION 14

Core Business Logic

Six features exist. Two are business features (F1, F2); the others are platform behaviour that every request or the whole process depends on.

F1 · User registration

Field	Details
Purpose	Let a person create credentials for later login
Files	routes/auth.routes.ts:16 · controllers/auth.controller.ts:8–23 · services/auth.service.ts:21–37 · repositories/user.repository.ts:17–32 · utils/app-error.ts
Trigger	POST /auth/register
Steps	1) controller passes req.body; 2) findByEmail; 3) found → AppError 409; 4) bcrypt.hash(password, 12); 5) create → UUID, push; 6) formatUser strips the hash; 7) 201 { message, data }
Data	Read: users array. Write: one User appended.
Dependencies	bcrypt (native, thread pool), global crypto
Success	201 with { id, name, email }
Failure	409 duplicate; 500 for a missing body or password; 403 CORS; 500 malformed JSON
Async	await on repository calls (resolve immediately) and on bcrypt.hash (real asynchronous work)
Example	{"name":"Ann","email":"a@x.io","password":"…"} → 201, id 2a038b70-… (§23, Example A)

F2 · Login and access-token issuance

Field	Details
Purpose	Authenticate credentials and hand out a bearer token for future protected APIs
Files	routes/auth.routes.ts:18 · controllers/auth.controller.ts:25–40 · services/auth.service.ts:39–73
Trigger	POST /auth/login
Steps	1) findByEmail; 2) missing → 401; 3) bcrypt.compare; 4) mismatch → same 401; 5) read JWT_ACCESS_SECRET (missing → Error → 500); 6) jwt.sign({ sub: id, email }, secret, { expiresIn: "15m" }); 7) 200 { accessToken, user }
Data	Read only
Dependencies	bcrypt, jsonwebtoken, environment
Success	200 with an HS256 token (900 s lifetime)
Failure	401 (identical message for unknown email and wrong password); 500 for configuration or input problems
Async	bcrypt.compare on the thread pool; jwt.sign is called synchronously (no callback)
Example	Valid credentials → 200 in ~167 ms; wrong password → 401 in ~163 ms; unknown email → 401 in ~0 ms

F3 · Request authentication middleware (implemented, unused)

Field	Details
Purpose	Guard future routes for users (JWT) and internal services (API key)

Field	Details
Files	middlewares/auth.middleware.ts; types/express.d.ts
Trigger	None today. It would run when added to a route, for example router.get(path, authenticate, handler).
Steps	Both headers → 400; x-api-key → timing-safe compare with INTERNAL_API_KEY → req.authType = "service"; Bearer → jwt.verify → req.authType = "user", req.user = { id: sub, email }; otherwise 401
Data	None; populates req fields only
Failure	400 / 401 JSON written directly (no requestId); missing secret or key → thrown Error → 500

F4 · Request correlation and access logging

Field	Details
Purpose	Trace each request across log lines and to the client
Files	middlewares/request-id.middleware.ts; middlewares/logger.middleware.ts; middlewares/error.middleware.ts
Steps	ID reused or generated → set on request and response → error log and access log both read it from the response header
Example	Client sends x-request-id: trace-123 → response header, error JSON and both log lines carry trace-123 (VERIFIED AT RUNTIME)

F5 · Cross-origin policy

Field	Details
Purpose	Allow only configured browser origins to call the API with credentials
Files	middlewares/cors.middleware.ts
Steps	Allowlist built at load → per request: no Origin or allowlisted → CORS headers + continue; else AppError 403
Example	Origin http://evil.test → 403 {"message":"Origin http://evil.test not allowed by CORS"}

F6 · Graceful shutdown

Field	Details
Purpose	Stop cleanly on deployment or Ctrl-C
Files	src/index.ts:38–87
Steps	Signal → log → server.close → log → (resource cleanup commented out) → exit 0; errors → exit 1

SECTION 15

External Integrations

> **NOT APPLICABLE**
> No external service is called at runtime. There are no outbound HTTP clients (fetch/axios), third-party SDKs, webhooks or OAuth providers. bcrypt and jsonwebtoken are local libraries, not network integrations.

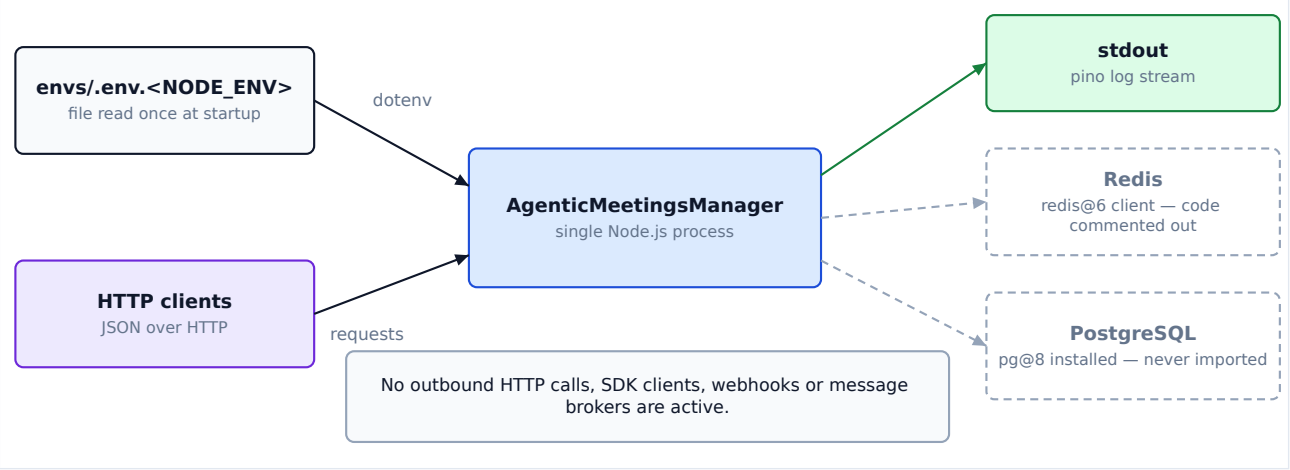


Figure 9 — External integration status. Only stdout logging and the env file are real I/O besides inbound HTTP; dashed targets are dormant.

Integration	Configuration	Client	Status
Redis	`REDIS_URL`	redis createClient (config/redis.ts, commented)	Disabled in the working tree. At commit 306d9c0 the server connected on start-up, but no code ever issued Redis commands.
PostgreSQL	–	pg (not imported)	Dependency only
Environment file	`envs/.env.<env>`	dotenv	Active, synchronous read at start-up
Log sink	–	pino → stdout	Active. In dev/qa through a pino-pretty worker thread. Shipping to a log platform is left to the hosting environment.

Because there are no integrations, the backend has no retry policies, timeouts, circuit breakers or reconciliation logic. These become necessary as soon as a database or a meeting-platform API is added (§22).

SECTION 16

Queues, Events, and Background Jobs

> **NOT APPLICABLE**
> No message queues, event buses, workers, cron or scheduled jobs exist. No queue or scheduler library is installed, the Redis client that could back one is commented out, and no module uses `setInterval` or worker threads.

The only asynchronous behaviours in the process are:

- bcrypt hash and compare, executed on libuv's thread pool and awaited inside the request.
- The `res.on('finish')` listener used for access logging.
- The pino-pretty transport worker thread (dev/qa only).
- SIGTERM/SIGINT handlers and the `server.close` callback.

SECTION 17

Authentication, Authorization, and Security

17.1 Authentication flow

A client registers, then logs in with email and password. The service checks the bcrypt hash and returns a signed JWT. The intended flow is that the client sends `Authorization: Bearer <token>` on later requests and `authenticate` validates it, but no route requires this yet, so the token currently cannot be used for anything.

Token property	Value	Evidence
Algorithm	HS256 (HMAC-SHA256, shared secret)	Runtime: decoded header
Claims	sub (user id), email, iat, exp	auth.service.ts:58–67; runtime
Lifetime	15 minutes (900 s)	auth.service.ts:65; runtime
Secret	JWT_ACCESS_SECRET (same for sign and verify)	auth.service.ts:52; auth.middleware.ts:62
Issuer / audience	Not set, not checked	Code
Refresh / revocation / logout	Not implemented	Code

17.2 authenticate middleware decision table

Request headers	Result	Line
x-api-key and Authorization both present	400 "Use either x-api-key or access token"	`auth.middleware.ts:18–24`
x-api-key, INTERNAL_API_KEY unset	Throws Error → 500 via error middleware	L33–35
x-api-key, wrong length or value	401 "Invalid API key"	L37–47
x-api-key matches	req.authType = "service"; next()	L49–52
Bearer token, JWT_ACCESS_SECRET unset	Throws Error → 500	L64–66
Bearer token valid	req.authType = "user"; req.user = { id: sub, email }; next()	L69–79
Bearer token invalid or expired	401 "Invalid or expired access token"	L80–86
Neither / non-Bearer Authorization	401 "Authentication required"	L93–95

There are no roles, permissions or ownership checks; authorization (as distinct from authentication) does not exist.

17.3 Verified security controls

Control	Implementation
Password hashing	bcrypt, cost factor 12; the hash is never returned (formatUser)
Generic login failure	Same message and 401 for unknown email and wrong password (auth.service.ts:43, 49)
Short-lived tokens	15-minute expiry
Constant-time API key check	Length check, then crypto.timingSafeEqual (auth.middleware.ts:37–39), in unused code

Control	Implementation
CORS allowlist	Explicit origins only, with credentials (cors.middleware.ts)
Internal error masking	Non-AppError messages replaced with "Internal server error"
Secrets kept out of git	.gitignore excludes envs/.env.*; env files are untracked
Request bodies not logged	Access and error logs carry method, URL, status and IDs only

17.4 Risks and potential vulnerabilities

ID	Issue	Evidence	Severity	Fix
S1	No brute-force or rate limiting on /auth/login and /auth/register	No limiter in package.json or code	High (prod)	R7
S2	No input validation; arbitrary JSON shapes reach the service and store	Runtime: 500s, 201 without email	Medium	R1
S3	Account enumeration: 409 on register, plus a ~165 ms timing gap on login	Runtime access logs: 0 ms vs 163 ms	Medium	R9
S4	Duplicate accounts through a concurrent-registration race	Runtime: 3/3 duplicates	Medium	R2
S5	No security headers (helmet); X-Powered-By: Express sent	Runtime response headers	Low	R10
S6	Client-controlled x-request-id accepted verbatim into logs and responses	request-id.middleware.ts:9–12	Low	R11
S7	jwt.verify does not pin algorithms, issuer or audience	auth.middleware.ts:69	Low	R12
S8	Missing secrets are detected only at request time, as a 500	auth.service.ts:52–56	Low	R6
S9	Email not normalised, so case variants create separate accounts	Runtime	Low	R2

Table 9 — Security findings (recommendations are listed in §22)

> **INFERENCE**
> On S7: jsonwebtoken 9 rejects `alg: none` and limits algorithms by key type, so a string secret only accepts HS* algorithms. Pinning `algorithms: ['HS256']` is defence in depth, not a fix for an exploitable bug.

17.5 Injection and data exposure

- **SQL/NoSQL injection:** not applicable. There are no queries; lookups are JavaScript strict equality on an array.
- **Log injection:** pino escapes values as JSON strings. The main exposure is unbounded or misleading request IDs (S6).
- **Error details:** messages are generic for unexpected errors, and stack traces are never sent to clients. However, the logging bug (§18.6) means stack traces are also missing from logs.
- **Sensitive data in logs:** the env-file path is logged; secrets and tokens are not.

SECTION 18

Error Handling and Logging

18.1 Error model

`AppError(message, statusCode)` in `src/utils/app-error.ts` is the only custom error class. Services throw it for expected outcomes. Controllers catch everything and call `next(error)`. `errorMiddleware` turns an AppError into its status and message and anything else into 500 with "Internal server error".

Condition	Raised at	Status	Response message	Verified
Email already registered	`auth.service.ts:25`	409	User already exists	runtime
Unknown email / wrong password	`auth.service.ts:43, 49`	401	Invalid email or password	runtime
Origin not allowlisted	`cors.middleware.ts:24`	403	Origin … not allowed by CORS	runtime
JWT_ACCESS_SECRET missing	`auth.service.ts:55`	500	Internal server error	code
Missing body / password	`TypeError / bcrypt`	500	Internal server error	runtime
Malformed JSON	`express.json()`	500	Internal server error (no requestId)	runtime
No matching route	`Express finalhandler`	404	HTML "Cannot GET …"	runtime

Table 10 — Error-to-response mapping

18.2 Logging

Aspect	Detail
Library / sink	pino → stdout. Pretty-printed via worker thread in dev/qa; JSON lines otherwise.
Levels in use	info (lifecycle, access log), warn (CORS config), error (request failures, shutdown errors), fatal (start-up failure). debug is enabled in dev/qa but never called.
Common fields	level, time (ISO-8601), service = agentic-meetings-manager, msg
Access-log fields	method, url, statusCode, duration ("Nms"), requestId
Error-log fields	error, requestId, method, url, statusCode
Correlation	x-request-id, reused from the client or generated

Actual production-format log lines (runtime, IDs shortened)

```
{"level":30,"time":"2026-10-26T05:33:15.991Z","service":"agentic-meetings-manager","method":"POST",
 "url":"/auth/register","statusCode":201,"duration":"173ms","requestId":"5683953d-…","msg":"HTTP reques
↪ t completed"}
{"level":50,"time":"2026-10-26T05:33:15.997Z","service":"agentic-meetings-manager","error":{"statusCode
↪ ":409},
 "requestId":"b609f59f-…","method":"POST","url":"/auth/register","statusCode":409,"msg":"Request failed
↪ "}
```

18.3 Health checks and monitoring

> **NOT APPLICABLE**
> There is no health or readiness endpoint, no metrics endpoint and no APM/tracing integration.

18.4 Retry and recovery

Nothing retries. Each request either succeeds or fails once. Process-level recovery (restarts) must come from the environment, for example a container orchestrator or process manager, none of which is configured here.

18.5 Debugging guide

Symptom	Where to look
Any error response	Search logs for the response's requestId and look at the 'Request failed' line in error.middleware.ts:19–28
500 with no detail in logs	The error was not an AppError and its message was dropped (§18.6). Set a breakpoint at error.middleware.ts:14 to inspect error.
401 on valid-looking login	auth.service.ts:40 (lookup, case-sensitive) and L46 (bcrypt.compare). Remember that users vanish on restart.
500 on login only	JWT_ACCESS_SECRET missing: auth.service.ts:52–56
Browser blocked / 403	CORS_ORIGINS value and the start-up warning; cors.middleware.ts:19
No access-log line	The request failed in express.json() or CORS, before loggerMiddleware

18.6 Known gaps (verified)

- Errors logged under `error` instead of `err`, so pino's serializer is bypassed and message and stack are lost.
- HTTP-aware errors from libraries (`status`/`statusCode` on body-parser errors) are flattened to 500.
- Responses to failures before `requestIdMiddleware` have no `requestId`.
- Early failures (JSON, CORS) produce an error log but no access log, so access-log metrics undercount failures.
- 404s bypass both JSON formatting and the error log.

SECTION 19

Testing and Debugging

19.1 Current state

> **NOT APPLICABLE**
> No automated tests exist. There are no test files, no test framework (Jest, Vitest, Mocha …), no `test` script, no test configuration and no coverage tooling. All modules are untested; the highest-value targets are AuthService, the error middleware and authenticate.

19.2 Static verification

```
npx tsc --noEmit        # exit code 0 on 2026-10-26 (strict mode, no type errors)
```

19.3 Manual smoke test used for this document

Reproducible without touching real configuration: `NODE_ENV=doccheck` points at a non-existent env file, so only the variables on the command line apply.

Smoke test script

```
NODE_ENV=doccheck PORT=4799 JWT_ACCESS_SECRET=local-doc-test \
CORS_ORIGINS=http://localhost:5173 npx tsx src/index.ts

B=localhost:4799
curl -i -X POST $B/auth/register -H 'content-type: application/json' \
     -d '{"name":"Ann","email":"a@x.io","password":"pw123"}'            # 201
curl -X POST $B/auth/register -H 'content-type: application/json' \
     -d '{"name":"Ann","email":"a@x.io","password":"pw123"}'            # 409
curl -X POST $B/auth/login -H 'content-type: application/json' \
     -d '{"email":"a@x.io","password":"pw123"}'                        # 200 + token
curl -X POST $B/auth/login -H 'content-type: application/json' \
     -d '{"email":"a@x.io","password":"nope"}'                         # 401
curl -X POST $B/auth/register                                          # 500
curl -X POST $B/auth/login -H 'content-type: application/json' -d '{bad'  # 500
curl $B/nope                                                           # 404 HTML
curl -X POST -H 'Origin: http://evil.test' $B/auth/login               # 403
curl -i -X OPTIONS -H 'Origin: http://localhost:5173' \
     -H 'Access-Control-Request-Method: POST' $B/auth/login             # 204
for i in 1 2 3; do curl -X POST $B/auth/register -H 'content-type: application/json' \
     -d '{"name":"R","email":"race@x.io","password":"pw"}' & done; wait  # 3 × 201
```

Check	Expected by design	Observed
Register new user	201	201, UUID id, no password in response
Register same email	409	409 + requestId
Login valid / invalid password	200 / 401	200 with HS256 token / 401
Register with no body	4xx	500 (no validation)
Register with no email	4xx	201 (user stored without email)
Malformed JSON	400	500, no requestId
Unknown route	JSON 404	HTML 404
Disallowed Origin / allowed preflight	403 / 204	403 JSON / 204 with CORS headers
3 concurrent same-email registrations	1 × 201, 2 × 409	3 × 201
Client x-request-id	echoed	echoed in header, body and logs
SIGTERM	graceful exit	3 shutdown log lines, clean exit

Table 11 — Smoke test results, 26 October 2026

19.4 Useful breakpoints

Location	What you can inspect
`src/index.ts:24`	Configured PORT and environment at listen time
`src/middlewares/request-id.middleware.ts:12`	Final request ID
`src/middlewares/cors.middleware.ts:19`	Origin decision
`src/controllers/auth.controller.ts:14 / :31`	Raw req.body entering the service
`src/services/auth.service.ts:22 / :40`	Lookup result for the email
`src/services/auth.service.ts:58`	Token claims before signing
`src/repositories/user.repository.ts:29`	Contents of the in-memory users array
`src/middlewares/error.middleware.ts:14`	Original error object, including message and stack

To debug in VS Code (RECOMMENDATION, not configured in the repository): start with `npx cross-env NODE_ENV=dev node --inspect --import tsx src/index.ts` and attach the debugger, or add a launch configuration that runs the same command. tsx provides source maps, so breakpoints in `.ts` files bind directly.

19.5 Troubleshooting

Symptom	Likely cause	Fix
Log shows 'Server started' with no port	PORT unset; server is on a random port	Set PORT in the env file or environment
Every browser call → 403	CORS_ORIGINS unset or origin mismatch (scheme and port must match exactly)	Add the exact origin; restart
Login → 500	JWT_ACCESS_SECRET missing	Set it; restart
Users disappear	In-memory store reset by restart or tsx watch reload	Expected until a database is added
Env values ignored	Server started outside the repository root, or variable already set in the shell	Run from the root; check the 'Loaded env file' log line
Logs are JSON in development	NODE_ENV set only in the env file, not in the process	Use npm run dev
node dist/index.js tries Redis	dist/ is stale	Run npm run build, or delete dist/
Port already in use → crash with stack trace	No server 'error' handler	Free the port or change PORT

19.6 Suggested test strategy (recommendation)

- Unit tests for AuthService with an in-memory repository: duplicate, hash never returned, 401 paths, token claims and expiry.
- HTTP tests (e.g. Vitest/Jest + supertest) against the exported app for status codes, the error envelope, CORS and request-ID propagation. This requires exporting `app` from a module separate from `listen`.
- A regression test for each verified defect in §2.2 before fixing it.

SECTION 20

Design Patterns and Code Quality

Pattern	Where	Notes
Layered architecture	`routes → controllers → services → repositories`	Clear one-way dependencies; HTTP concerns stay in controllers
Constructor injection (manual)	`auth.routes.ts:9–13`	Testable by passing fakes; no container. The service depends on a concrete class, not an interface.
Repository	`repositories/user.repository.ts`	Async API that is ready for a real database
Middleware pipeline	`index.ts:13–18`	Ordered cross-cutting concerns; error middleware last
Custom error type	`utils/app-error.ts`	Separates expected (AppError) from unexpected errors
Composition root	`index.ts and auth.routes.ts`	Wiring is split between two files
Module augmentation	`types/express.d.ts`	Typed req.user / req.authType
DTO shaping	`AuthService.formatUser`	Prevents the password hash from leaking

Table 12 — Patterns supported by the code

20.1 Strengths

- Small, readable modules with one responsibility each; strict TypeScript with unused-code checks; consistent naming (`*.middleware.ts`, `*.service.ts`).
- Security-conscious defaults: bcrypt cost 12, generic login errors, timing-safe key comparison, CORS allowlist, masked 500 messages.
- Observability foundations exist early: structured logs, request IDs and graceful shutdown.

20.2 Weaknesses and maintenance risks

- Dead or dormant code: auth and not-found middleware, commented Redis module and imports, unused dependencies (pg, redis, zod).
- Compile-time types stand in for runtime validation.
- Configuration is spread across modules and read lazily, with no single validated config object.
- `app` creation and `listen` are in the same module, which makes HTTP testing harder.
- Stale build artefacts (`dist/`) and an empty README.
- The divergence between branches (§3.3) is a process risk for upcoming merges.

SECTION 21

Performance and Scalability Considerations

- **bcrypt dominates latency.** At cost 12, hash and compare took about 163–173 ms each locally. They run on libuv's thread pool, which has 4 threads by default, so at most 4 hashes run in parallel per process (INFERENCE from Node defaults). Register and login throughput is bounded by this.
- **User lookups are O(n).** `Array.find` over all users is fine at small scale and is replaced by an indexed query once a database exists.
- **Single process, in-memory state.** The service cannot scale horizontally without shared storage, because each instance would have its own users (§13.4).
- **Unbounded memory.** The users array never shrinks.
- **Development tooling in production.** `npm run prod` runs through tsx, which transpiles at start-up. Running compiled `dist/` with plain node would start faster and avoid the dev dependency (RECOMMENDATION).
- **Logging cost.** JSON logs to stdout in production are cheap. pino-pretty runs off the main thread in dev/qa.

> **RECOMMENDATION**
> Before load testing, add a database with a unique email index, rate limiting on auth routes, and run compiled JavaScript. Tune `UV_THREADPOOL_SIZE` only if bcrypt queueing shows up in measurements.

SECTION 22

Known Limitations and Recommendations

Everything in this section is a recommendation, not existing functionality. Priorities: P1 = correctness or security issue already observable, P2 = needed before production, P3 = maintainability.

ID	Pri	Recommendation	Addresses
R1	P1	Validate request bodies with zod schemas (email format, required fields, length limits) in a small validate middleware or in the controller	§2.2 #1, S2; uses an existing dependency
R2	P1	Normalise email (trim, lower-case) and enforce uniqueness at the storage layer (unique index), mapping violations to 409	Race in §13.3, S4, S9
R3	P1	Error middleware: log with err key; honour status/ statusCode of 4xx library errors (400 for JSON, 413 for size)	§18.6
R4	P1	Mount notFoundMiddleware before errorMiddleware	HTML 404s
R5	P1	Reconcile branches: restore 306d9c0's changes on master/dev (revert the merge or cherry-pick)	§3.3
R6	P2	Validate configuration at start-up (PORT, JWT_ACCESS_SECRET, CORS_ORIGINS) and fail fast; add envs/.env.example	§8, S8
R7	P2	Rate-limit and add lockout or back-off for /auth/login and /auth/register	S1
R8	P2	Replace the in-memory repository with PostgreSQL (or decide otherwise), keeping the repository interface; add migrations	§13
R9	P2	Reduce enumeration: dummy bcrypt compare for unknown users; consider a neutral register response	S3
R10	P2	Add helmet (or equivalent) and app.disable('x-powered-by')	S5
R11	P3	Accept client x-request-id only if it matches a safe pattern and length; otherwise generate one	S6
R12	P3	Pin algorithms (and later issuer/audience) in jwt.verify; plan refresh tokens and revocation	S7
R13	P2	Add a test framework and the tests from §19.6; add a test script and CI	§19
R14	P2	Add a start script for compiled output (node dist/ index.js), delete the stale dist/, and add a Dockerfile and health endpoint	§7.7, §18.3
R15	P3	Remove or re-enable dormant code deliberately (Redis, pg, auth middleware); define a repository interface for DI	§20.2
R16	P3	Handle server 'error' events; add a forced-exit timeout to shutdown	§10.2, §10.3
R17	P3	Write README.md: purpose, setup, env vars, scripts	Onboarding

Table 13 — Prioritised recommendations

SECTION 23

End-to-End Execution Examples

All examples were executed against the local server on 26 October 2026. Tokens are redacted and request IDs shortened.

Example A — Register, then log in

1. `POST /auth/register` with name Ann, email a@x.io. Passes JSON parsing, gets a request ID, has no Origin header (allowed by CORS) and starts the access-log timer.
2. `AuthController.register` → `AuthService.register`: `findByEmail` returns undefined; `bcrypt.hash` takes about 170 ms; `create` assigns id 2a038b70-… and pushes the user.
3. Response 201 `{ message: "User registered successfully", data: { id, name, email } }`. Access log: statusCode 201, duration 173ms.
4. `POST /auth/login` with the same credentials → lookup hits → `bcrypt.compare` returns true → `jwt.sign` with sub = 2a038b70-…, email, exp = iat + 900.
5. Response 200 with `accessToken` and the user. Access log: 200, 167ms.

Example B — Duplicate registration

```
→ POST /auth/register {"name":"Ann","email":"a@x.io","password":"…"}       (second time)
  auth.service.ts:22  findByEmail → existing user
  auth.service.ts:25  throw new AppError("User already exists", 409)
  auth.controller.ts:21  next(error)  →  error.middleware.ts:14  statusCode 409
← 409 {"message":"User already exists","requestId":"b609f59f-…"}
log  level 50 "Request failed" statusCode 409   ·   level 30 "HTTP request completed" 0ms
```

Example C — Malformed JSON

```
→ POST /auth/login   Content-Type: application/json   body: {bad
  express.json() → SyntaxError (type entity.parse.failed, status 400)
  requestIdMiddleware / cors / logger never run
  error.middleware.ts:14 → not an AppError → 500
← 500 {"message":"Internal server error"}            (no requestId field)
log  level 50 "Request failed" error.status 400, statusCode 500   (no access-log line)
```

Example D — CORS

```
→ POST /auth/login   Origin: http://evil.test
  cors.middleware.ts:24 → AppError("Origin http://evil.test not allowed by CORS", 403)
← 403 {"message":"Origin http://evil.test not allowed by CORS","requestId":"9a36f2ea-…"}

→ OPTIONS /auth/login   Origin: http://localhost:5173   Access-Control-Request-Method: POST
← 204  Access-Control-Allow-Origin: http://localhost:5173
       Access-Control-Allow-Credentials: true
       Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
       Access-Control-Allow-Headers: Content-Type,Authorization,x-request-id
```

Example E — Caller-supplied correlation ID

```
→ POST /auth/login   x-request-id: trace-123   body: {}
  findByEmail(undefined) → no user → AppError 401
← 401  x-request-id: trace-123   {"message":"Invalid email or password","requestId":"trace-123"}
log  "Request failed" requestId trace-123 · "HTTP request completed" requestId trace-123 0ms
```

SECTION 24

Glossary

Term	Meaning in this repository
AppError	Error subclass with an HTTP statusCode, used for expected failures (src/utils/app-error.ts)
bcrypt cost factor	Work factor (12 here); each +1 doubles hashing time
Bearer token	Authorization: Bearer <JWT> header format checked by authenticate
Composition root	The place where objects are constructed and wired: auth.routes.ts
CORS	Browser mechanism controlling which web origins may call the API
cross-env	Tool that sets environment variables in npm scripts on any OS
dotenv	Loads KEY=value pairs from a file into process.env
Error middleware	Express handler with four parameters (err, req, res, next) that runs after an error
Ephemeral port	Random free port the OS assigns when PORT is not given
HS256	HMAC-SHA256 JWT signature using one shared secret
JWT	JSON Web Token; signed claims (sub, email, iat, exp) returned at login
Module augmentation	TypeScript technique adding fields to Express.Request (types/express.d.ts)
pino / pino-pretty	Structured JSON logger / its human-readable development formatter
Request ID	Per-request correlation value carried in x-request-id
Timing-safe comparison	Comparison whose duration doesn't reveal how many characters matched
tsx	Runs TypeScript directly by transpiling on the fly (no type-check)
x-api-key	Header for service-to-service authentication in authenticate

SECTION 25

Appendix: Commands, Configuration, and File References

A. Command reference

```
npm install                          # install dependencies
npm run dev | qa | prod              # run (see §7.2)
npm run build                        # compile to dist/
npx tsc --noEmit                     # type-check
git diff 719ee6f origin/master       # empty → master lacks the auth feature (§3.3)
git status --short                   # working-tree changes described in §3.4
```

B. Mermaid sources for the diagrams

The figures in this PDF are drawn as vector graphics. The equivalent Mermaid definitions below can be pasted into any Mermaid renderer or a Markdown file in the repository.

Figure 1

```
%% Figure 1 — High-level architecture
flowchart LR
  ENV[envs/.env.NODE_ENV] -->|dotenv| APP
  C[HTTP client] --> J
  subgraph APP[Express app - src/index.ts]
    J[express.json] --> RID[requestIdMiddleware] --> CORS[corsMiddleware]
    CORS --> LOG[loggerMiddleware] --> R[Router /auth]
    R -->|next err| ERR[errorMiddleware]
  end
  R --> CTRL[AuthController] --> SVC[AuthService] --> REPO[UserRepository] --> MEM[(users array)]
  ERR -.error JSON.-> C
  APP -.-> P[pino -> stdout]
  REDIS[Redis client - commented out]:::off
  PG[pg - never imported]:::off
  AUTH[authenticate - not mounted]:::off
  classDef off stroke-dasharray: 4 3,color:#64748b
```

Figure 2

```
%% Figure 2 — Module dependency graph
flowchart TD
  index[src/index.ts] --> env[config/env.ts] --> logger[config/logger.ts]
  index --> logger
  index --> routes[routes/auth.routes.ts]
  index --> rid[request-id.middleware] & cors[cors.middleware]
  index --> lmw[logger.middleware] & emw[error.middleware]
  routes --> ctrl[auth.controller] & svc[auth.service] & repo[user.repository]
  ctrl --> svc --> repo
  svc --> apperr[utils/app-error]
  cors --> apperr
  emw --> apperr
  cors & lmw & emw --> logger
```

Figure 3

```
%% Figure 3 — Startup sequence
sequenceDiagram
  participant S as npm/tsx
  participant I as index.ts
  participant E as config/env.ts
  participant L as config/logger.ts
  participant R as auth.routes.ts
  participant C as cors.middleware.ts
```

```
  participant H as http.Server
  S->>I: load (NODE_ENV via cross-env)
  I->>E: import (L1)
  E->>L: import logger -> pino()
  E->>E: dotenv.config(envs/.env.<env>)
  I->>R: import -> new Repository/Service/Controller
  I->>C: import -> read CORS_ORIGINS
  I->>I: app.use x6
  I->>H: app.listen(PORT)
  H-->>L: 'Server started'
  I->>I: SIGTERM/SIGINT handlers
```

Figure 4

```
%% Figure 4 — Graceful shutdown
flowchart TD
  S[SIGTERM / SIGINT] --> A[shutdown: log signal] --> B[server.close]
  B --> C[log 'HTTP server closed'] --> D[Redis/DB cleanup - commented out]
  D --> E[log completed -> exit 0]
  D -.error.-> F[log error -> exit 1]
```

Figure 5

```
%% Figure 5 — Request lifecycle
flowchart TD
  Q[request] --> J[express.json] -->|ok| RID[requestId] --> CO[cors]
  CO -->|allowed| LG[logger] --> RT{route?}
  RT -->|/auth/*| CT[controller -> service -> repository] --> OK[201/200 JSON]
  RT -->|none| NF[Express default 404 HTML]
  J -->|bad JSON| EM[errorMiddleware]
  CO -->|rejected 403| EM
  CT -->|throw| EM --> EJ[JSON message, requestId]
```

Figure 6

```
%% Figure 6 — Register
sequenceDiagram
  Client->>Router: POST /auth/register
  Router->>Controller: register(req,res,next)
  Controller->>Service: register(req.body)
  Service->>Repository: findByEmail(email)
  Repository-->>Service: User | undefined
  alt exists
    Service-->>Controller: throw AppError 409
  else new
    Service->>bcrypt: hash(password, 12)
    Service->>Repository: create({name,email,hash})
    Repository-->>Service: User (randomUUID)
    Service-->>Controller: {id,name,email}
    Controller-->>Client: 201
  end
```

Figure 7

```
%% Figure 7 — Login
sequenceDiagram
  Client->>Controller: POST /auth/login
  Controller->>Service: login(req.body)
  Service->>Repository: findByEmail(email)
  alt not found
    Service-->>Controller: throw AppError 401
  else found
    Service->>bcrypt: compare(password, hash)
    alt mismatch
      Service-->>Controller: throw AppError 401
    else match
      Service->>jsonwebtoken: sign({sub,email}, JWT_ACCESS_SECRET, 15m)
      Service-->>Controller: {accessToken, user}
```

```
      Controller-->>Client: 200
    end
  end
```

Figures 8 and 9

```
%% Figure 8 — Data model
erDiagram
  USER {
    string id "crypto.randomUUID()"
    string name
    string email "unique via service check only"
    string password "bcrypt hash"
  }

%% Figure 9 — Integrations
flowchart LR
  ENV[env file] --> APP[AgenticMeetingsManager]
  CL[HTTP clients] --> APP
  APP --> OUT[stdout logs]
  APP -.disabled.-> REDIS[(Redis)]
  APP -.unused dependency.-> PG[(PostgreSQL)]
```

C. Inspection checklist

Path	Inspected	How
`src/**/*.ts (16 files, 572 lines)`	Yes, fully	Read every line; traced imports; type-checked; exercised at runtime
`package.json, tsconfig.json, .gitignore`	Yes, fully	Read
`.vscode/settings.json`	Yes, fully	Read
`README.md, prisma/schema.prisma, docs/`	Yes	Empty (0 bytes / empty directory)
`package-lock.json`	Partially	lockfileVersion only; installed versions read from node_modules
`dist/index.js, dist/config/redis.js`	Yes	Compared with source to establish dist/ is stale
`Other dist/*.js`	Listed only	Generated output; not documented
`envs/.env.dev, envs/.env.qa`	Existence only	Not opened (secrets)
`node_modules/`	Versions only	Third-party code not documented
`Git history and branches`	Yes	log --all, branch -a, diff against origin/master and origin/dev

D. Items that could not be verified

- The variables and values actually set in `envs/.env.dev` and `envs/.env.qa`, including whether PORT, JWT_ACCESS_SECRET and CORS_ORIGINS are defined there.
- How and where the service is deployed in practice; no deployment artefacts exist.
- The intended database and schema (PostgreSQL/Prisma is only suggested by dependencies).
- The planned meeting and agent features. No code, documentation or README describes them.
- Behaviour on Node versions other than v24.15.0.
- Behaviour of the listen-error and oversize-body paths, which were reasoned about but not triggered.